

Seamless, Correct, and Generic Programming over Serialised Data

GUILLAUME ALLAIS

University of Strathclyde, G1 1XH
(e-mail: guillaume.allais@ens-lyon.org)

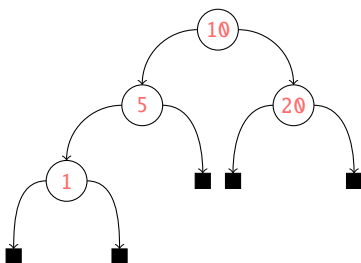
Abstract

In typed functional languages, one can typically only manipulate data in a type-safe manner if it first has been deserialised into an in-memory tree internally represented as a graph of nodes-as-structs and subterms-as-pointers. We demonstrate how we can use Quantitative Type Theory as implemented in the dependently typed programming language Idris 2 to define a small universe of serialised datatypes, and provide generic programs allowing users to process values stored contiguously in buffers. The code manipulating buffer-bound values is extremely similar to its pure counterpart processing inductive structures thus allowing a seamless user experience. Our approach allows implementors to prove the full functional correctness by construction of the IO functions processing the data stored in the buffer. We finally observe how this approach gives us a significant speedup for functions that do not need to explore the full tree.

This work has been implemented in Idris 2 and fully ported to Agda, allowing programs written in the two languages to easily exchange data.

1 Introduction

In (typed) functional language we are used to manipulating structured data by pattern-matching on it. We include an illustrative example below.



```
data Tree
  = Leaf
  | Node Tree Bits8 Tree

sum : Tree -> Nat
sum t = case t of
  Leaf => 0
  Node l b r =>
    let m = sum l
        n = sum r
    in (m + cast b + n)
```

On the left, an example of a binary tree storing bytes in its nodes and nothing at its leaves. On the right, a small Idris 2 snippet declaring the corresponding inductive type `Tree` with two constructors `Leaf` and `Node` and defining a function `sum` adding up all of the nodes' contents. It proceeds by pattern-matching on its input `t`: if the tree is a leaf then we immediately return 0, otherwise we start by recursively summing up the left and right

subtrees, cast the byte to a natural number and add everything up. Simply by virtue of being accepted by the typechecker, we know that this function is covering (all the possible patterns have been handled) and total (all the recursive calls are performed on smaller trees).

Idris Specifics 1 (Semantic Highlighting). *All of the code snippets in this paper are semantically highlighted using Katla (Idris Community, 2023). This means that they all have been successfully parsed and scope-and-type checked. Different colours are used for keywords, data constructors, function definitions, types, unsafe postulates, and bound variables.*

At runtime, the tree will quite probably be represented by constructors-as-structs and substructures-as-pointers: each constructor will be a struct with a tag indicating which constructor is represented and subsequent fields will store the constructors' arguments. Each argument will either be a value (e.g. a byte) or a pointer to either a boxed value or a substructure. If we were to directly write a function processing a value in this encoding, proving that a dispatch over a tag is covering, and that the pointer-chasing is terminating relies on global invariants tying the encoding to the inductive type. Crucially, the functional language allows us to ignore all of these details and program at a higher level of abstraction where we can benefit from strong static guarantees.

Unfortunately not all data comes structured as inductive values abstracting over a constructors-as-structs and substructures-as-pointers runtime representation. Data that is stored in a file or received over the network is typically represented in a contiguous binary format with a high information density.

We include below a textual representation of the above tree using node and leaf constructors and highlighting the data in red.

```
(node (node (node leaf 1 leaf) 5 leaf) 10 (node leaf 20 leaf))
```

This looks almost exactly like the list of bytes we get when using a naïve serialisation format based on a left-to-right in-order traversal of this tree. In the encoding below, leaves are represented by the byte 00, and nodes by the byte 01 (each byte is represented by two hexadecimal characters, we have additionally once again highlighted the bytes corresponding to data stored in the nodes):

```
(node (node leaf 1 leaf) 5 leaf)
01 01 01 00 01 00 05 00 0a 01 00 14 00
      (node leaf 1 leaf)
```

The idiomatic way to process such data in a functional language is to first deserialise it as a value of the inductive type `Tree` and then call the tree-processing `sum` function we defined above. If we were using a lower-level language however, we could directly process the serialised data without the need to fully deserialise it. Even a naïve port of `sum` to C can indeed work directly over buffers:

```
1 int sumAt (uint8_t buf[], int *ptr) {
2   uint8_t tag = buf[*ptr]; (*ptr)++;
```

```

3  switch (tag) {
4      case 0: return 0;
5      case 1:
6          int m = sumAt(buf, ptr);
7          uint8_t b = buf[*ptr]; (*ptr)++;
8          int n = sumAt(buf, ptr);
9          return (m + (int) b + n);
10     default: exit(-1); }

```

This function takes a buffer of bytes, and a pointer currently indicating the start of a tree and returns the corresponding sum. We start (line 2) by reading the byte the pointer is referencing and immediately move the pointer past it. This is the tag indicating which constructor is at the root of the tree and so we inspect it (line 3). If the tag is 0 (line 4), the tree is a leaf and so we return 0 as the sum. If the tag is 1 (line 5), then the tree starts with a node and the rest of the buffer contains first the left subtree, then the byte stored in the node, and finally the right subtree. We start by summing the left subtree recursively (line 6), after which the pointer has been moved past its end and is now pointing at the byte stored in the node. We can therefore dereference the byte and move the pointer past it (line 7), compute the sum over the right subtree (line 8), and finally add up all the components, not forgetting to cast the byte to an int (line 9). If the tag is anything other than 0 or 1 (line 10) then the buffer does not contain a valid tree and so we immediately exit with an error code.

As we can readily see, this program directly performs pointer arithmetic, explicitly mentions buffer reads, and relies on undocumented global invariants such as the structure of the data stored in the buffer, or the fact that the pointer is being moved along and points directly past the end of a subtree once `sumAt` has finished computing its sum.

Our goal with this work is to completely hide all of these dangerous aspects and offer the user the ability to program over serialised data just as seamlessly and correctly as if they were processing inductive values. We will see that Quantitative Type Theory (QTT) (McBride, 2016; Atkey, 2018) as implemented in Idris 2 (Brady, 2021) empowers us to do just that purely in library code.

1.1 Seamless Programming over Serialised Data

Forgetting about correctness for now, the seamlessness of our approach can be summed up by the following code snippet in which we compute the sum of the bytes present in a binary tree stored in a buffer.

```

sum : Pointer.Mu Tree _ -> IO Nat
sum ptr = case !(view ptr) of
  "Leaf" # _ => pure Z
  "Node" # l # b # r =>
    do m <- sum l
       n <- sum r
       pure (m + cast b + n)

```

We reserve for later our detailed explanations of the concepts used in this snippet (`Pointer.Mu` in Section 6, `view` in Section 8.4). For now, it is enough to understand that the

function is an `IO` process inspecting a buffer that contains a tree stored in serialised format and computing the same sum as the pure function seen in the previous section. In both cases, if we uncover a leaf ("`Leaf`" # `_`) then we return zero, and if we uncover a node ("`Node`" # `l` # `b` # `r`) with a left branch `l`, a stored byte `b`, and a right branch `r`, then we recursively compute the sums for the left and right subtrees, cast the byte to a natural number and add everything up. Crucially, this function and its pure counterpart defined in the previous section look eerily similar, thanks to the fact that the one operating on serialised data uses high level combinators and does not explicitly perform error-prone pointer arithmetic, or low-level buffer reads.

This seamlessness is the first way in which our approach shines.

1.2 Correct Programming over Serialised Data

One major difference between the two functions seen above is that we can easily prove some of the pure function's properties by a structural induction on its input whereas we cannot prove anything about the `IO` process without first explicitly postulating the `IO` monad's properties. We will see that we can instead refine that second definition to obtain a correct-by-construction version of `sum`, with almost exactly the same code.

```
sum : Pointer.Mu Tree t ->
      IO (Singleton (Data.sum t))
sum ptr = case !(view ptr) of
  "Leaf" # _ => pure [! Z !]
  "Node" # l # b # r =>
    do m <- sum l
       n <- sum r
    pure [! [! m + [! cast b !] !] + n !]
```

In the above snippet, we can see that the `Pointer.Mu` is indexed by a phantom parameter: a runtime irrelevant `t` which has type `(Data.Mu Tree)`. And so the return type can mention the result of the pure computation `(Data.sum t)`. `Singleton` is, as its name suggests, a singleton type (cf. [Section 7](#)) i.e. it is a wrapper around a natural number that is proven to be equal to the one computed by the pure `sum` function. And so we can transfer any property proven on the pure `sum` to the one operating on values residing in buffers.

The implementation itself only differs in that we had to use idiom brackets (McBride and Paterson, 2008), something we will explain in [Section 7.2](#).

In other words, our approach also allows us to, *at the same time*, define and prove the functional correctness of the `IO` procedures processing trees stored in serialised format in a buffer. This means in particular that any intermediate computation can rely on the fact that the recursive calls are already known to be correct.

This intrinsic correctness is our second main contribution.

1.3 Generic Programming over Serialised Data

Last but not least, as Altenkirch and McBride (2002) demonstrated: “With dependently (sic) types, generic programming is just programming: it is not necessary to write a new compiler each time a useful universe presents itself.”

In this paper we carve out a universe of inductive types that can be uniformly serialised and obtain all of our results by generic programming. In practice this means that we are not limited to the type of binary trees with bytes stored in the nodes we used in the examples above. We will for instance be able to implement a generic and correct-by-construction definition of `fold` operating on data stored in a buffer whose type declaration can be seen below (we will explain the pure generic fold in Section 3.3 and define its counterpart operating on buffers in Section 8.5).

```
fold : {cs : Data nm} -> (alg : Alg cs a) ->
      forall t. Pointer.Mu cs t ->
      IO (Singleton (Data.fold alg t))
```

This data-genericity is our third contribution.

1.4 Plan

In summary, we are going to define a library for the seamless, correct, and generic manipulation of algebraic types in serialised format.

Section 2 introduces the language of descriptions capturing the subset of inductively defined types that our work can handle. It differs slightly from usual presentations in that it ensures the types can be serialised and tracks crucial invariants towards that goal. Section 3 gives a standard meaning to these data descriptions as strictly positive endofunctors whose fixpoints give us the expected inductive types. We will use this standard meaning in the specification layer of our work. Section 4 explores the serialisation format we have picked for these trees: a depth-first, left-to-right infix traversal of the trees, with additional information stored to allow for the direct access of any subtree. Section 6 defines the type of pointers to trees stored in a buffer and shows how we can use such pointers to write the corresponding tree to a file. Section 7 introduces the terminology of *views* and *singleton* types that is crucial to the art of programming in a correct-by-construction manner. Section 8 defines IO primitives that operate on serialised trees stored in an underlying buffer. They encapsulate all the unsafe low-level operations and offer a high-level interface that allows users to implement correct-by-construction procedures. Section 9 defines a set of serialisation combinators that allows users to implement correct-by-construction procedures writing values into a buffer. Section 10 discusses some preliminary performance results for the library.

2 Our Universe of Descriptions

We first need to pin down the domain of our discourse. To talk generically about an entire class of datatypes without needing to modify the host language we have decided to perform a universe construction (Benke et al., 2003; Morris, 2007; Löh and Magalhães, 2011). That

is to say that we are going to introduce an inductive type defining a set of codes together with an interpretation of these codes as bona fide host-language types. We will then be able to program generically over the universe of datatypes by performing induction on the type of codes (Pfeifer and Rueß, 1999).

The universe we define is in the tradition of a sums-of-products vision of inductive types (Jansson and Jeuring, 1997). In our setting, constructors are essentially arbitrarily nested tuples of values of type unit, bytes, and recursive substructures. A datatype is given by listing a choice of constructors.

2.1 Interlude: Serialisation Formats

Before giving a precise definition of the descriptions, let us think a little bit about the features we would like to see in our serialisation format and see how this informs our design. We reproduce below the naïve encoding of a binary tree with bytes stored in its nodes we gave in the introduction. It has both strengths and weaknesses.

(node (node leaf 1 leaf) 5 leaf)
 01 01 01 00 01 00 05 00 0a 01 00 14 00
 (node leaf 1 leaf)

One of its strength is the flattened encoding of nodes: the left subtree, byte stored, and right subtree are all stored contiguously after the tag 01 announcing that a node is present. This approach can be generalised to any type with a unique constructor: there is no need to store a byte corresponding to the constructor for the unit type, or for the constructor to a pair type. These unambiguous constructors can be reinserted during the type-directed decoding phase. In practice this means that our encoding will not retain the nesting structure described by a type. Concretely, $((a, b), c)$ and $(a, (b, c))$ will have the same representations: the encoding of a , followed by the encoding of b and last that of c . Similarly, values of type $()$ will be eluded entirely: values of type $((), a, ())$ and a will have the same serialised representation, the encoding of the value of type a .

The major weakness of the naïve format we presented above is the inability to process a node's right subtree without having first processed its left subtree. This is due to the fact that the subtree's size is not statically known and that the serialisation format does not store an offset for it. Consequently, if we wanted to fetch the rightmost byte stored in the tree, we would have to traverse the entire encoding. That represents an exponential slow-down compared to what we could achieve with a different encoding allowing for random access.

Based on these observations, we envision a serialisation format that will permit random access to all of the arguments of a node. Such a format will have to store an offset for all of a node's arguments whose size is not statically known and which we may want to jump past i.e. all of them except a constructor's very last argument.

2.2 Descriptions

This leads us to the inductive family `Desc` declared below as the type of constructor descriptions.

```

277 data Desc : (rightmost : Bool) ->
278           (static : Nat) -> (offsets : Nat) ->
279           Type

```

Idris Specifics 2 (Separate Declaration and Definition). *Idris 2 lets us declare a datatype or function first and only define it later. This is convenient for blocks of mutually-defined types and functions but can also be used, like here, to focus on the type signature first.*

This family has three indices corresponding to three crucial invariants being tracked. First, an index telling us whether the current description is being used in the `rightmost` branch of the overall constructor description. Second, the `statically` known size of the described data in the number of bytes it occupies. Third, the number of `offsets` that need to be stored to compensate for subterms not having a statically known size. The reader should think of `rightmost` as an ‘input’ index (the context in which the family is used tells it whether it is in a rightmost branch) whereas `static` and `offsets` are ‘output’ indices (the family’s own constructors each determine their respective sizes).

Next we define the family proper by giving its four constructors.

```

294 data Desc where
295   None : Desc r 0 0
296   Byte : Desc r 1 0
297   Prod : {sl, sr, ol, or : Nat} ->
298         Desc False sl ol -> Desc r sr or ->
299         Desc r (sl + sr) (ol + or)
300   Rec : Desc r 0 (ifThenElse r 0 1)

```

Each constructor can be used anywhere in a description so their index tracking whether we are in the rightmost branch can be an arbitrary boolean `r`.

`None` is the description of values of type unit. The static size of these values is zero as no data is stored in a value of type unit. Similarly, they do not require an offset to be stored as we statically know their size.

`Byte` is the description of bytes. Their static size is precisely one byte, and they do not require an offset to be stored either.

`Prod` gives us the ability to pair two descriptions together. As explained earlier pairs have a unique constructor and so their encoding will not consume any additional bytes. Consequently their static size and number of offsets are the respective sums of the static sizes and numbers of offsets of each subdescription. The description of the left element of the pair will never be in the rightmost branch of the overall constructors description and so its index is `False` while the description of the right element of the pair is in the rightmost branch precisely whenever the whole pair is; hence the propagation of the `r` arbitrary value from the return index into the description of the right component.

Last but not least, `Rec` is a position for a subtree. We cannot know its size in bytes statically and so we decide to store an offset unless we are in the rightmost branch of the overall description. Indeed, there are no additional constructor arguments behind the rightmost one and so we have no reason to skip past the subterm. Consequently we do not bother recording an offset for it.

2.3 Constructors

We represent a constructor as a record packing together a name for the constructor, the description of its arguments (which is, by virtue of being used at the toplevel, in rightmost position), and the values of the `static` and `offsets` invariants.

```
record Constructor (nm : Type) where
  constructor (::)
  name : nm
  {static : Nat}
  {offsets : Nat}
  description : Desc True static offsets
```

Idris Specifics 3 (Implicit Record Fields). *Record fields whose type declaration is surrounded by curly braces are implicit: they do not need to be explicitly mentioned when constructing the record, or when pattern-matching on it.*

Here the two invariants `static` and `offsets` are stored as implicit fields because as ‘output’ indices of the `Desc` family their value is easily reconstructed using unification (Abel and Pientka, 2011). Note that we used `(::)` as the name of the constructor for records of type `Constructor`. This allows us to define constructors by forming an expression reminiscent of Haskell’s type declarations: `name :: type`. Returning to our running example, this gives us the following encodings for leaves that do not store anything and nodes that contain a left branch, a byte, and a right branch.

```
Leaf : Constructor String
Leaf = "Leaf" :: None
```

```
Node : Constructor String
Node = "Node" :: Prod Rec (Prod Byte Rec)
```

2.4 Datatypes

A datatype description `Data` is given by a vector (also known as a length-indexed list) named `constructors` and containing constructor descriptions. For convenience, we store the number `consNumber` of constructors separately so that it does not need to be recomputed every time it is needed. We additionally insist that we have a proof `fitsInBits8` that the datatype has less than 255 constructors which will allow us to safely store the corresponding tag in a single byte. This is enforced using `So`, a type family ensuring the boolean check it is indexed by has succeeded.

```
record Data (nm : Type) where
  constructor MkData
  {consNumber : Nat}
  {auto 0 fitsInBits8 : So (consNumber <= 255)}
  constructors : Vect consNumber (Constructor nm)
```

Idris Specifics 4 (Auto Implicit Record Fields). *Implicit record fields whose type declaration is preceded by the `auto` keyword are auto implicit: just like implicit fields they do not need*

to be explicitly mentioned when constructing the record, or when pattern-matching on it. Additionally, these values are not constructed by unification but via a builtin type-directed proof search.

We can then encode our running example as a simple `Data` declaration: a binary tree whose node stores bytes is described by the choice of either a `Leaf` or `Node`, as defined above. The proof `fitsInBits8` is constructed automatically: the boolean test `(2 <= 255)` has trivially computed to `True`.

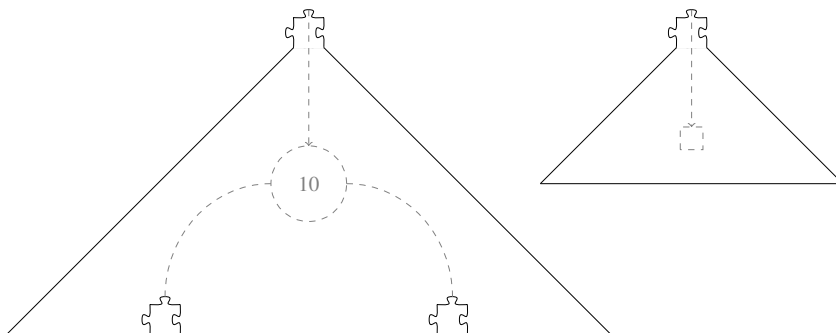
```
Tree : Data String
Tree = MkData [Leaf, Node]
```

Now that we have a language that allows us to give a description of our inductive types, we are going to give these descriptions a meaning as trees.

3 Meaning as Trees

We now see descriptions as functors and, correspondingly, datatypes as the initial objects of the associated functor-algebras. This is a standard construction derived from Malcolm’s work (1990), itself building on Hagino’s categorically-inspired definition of a lambda calculus with a generic notion of datatypes (Hagino, 1987).

Intuitively, the meaning of a description is the shape of one “layer” of tree where the subtrees’ meaning is left abstract. We include below two sketches showing what the respective meanings of the `Node` and `Leaf` descriptions look like. We use jigsaw pieces to figure ports in which subtrees can be inserted.



In our work these trees will be used primarily to allow users to give a precise specification of the IO procedures they actually want to write in order to process values stored in buffers. We expect these inductive trees and the associated generic programs consuming them to be mostly used at the $\emptyset$ modality i.e. to be erased during compilation.

3.1 Descs as Functors

We are going to define the **Meaning** of descriptions as strictly positive endofunctors on **Type** by induction on said descriptions. In its type, all of **r**, **s**, and **o** are implicitly universally quantified, a feature of Idris 2 we explain below.

```
Meaning : Desc r s n -> Type -> Type
```

Idris Specifics 5 (Implicit Prenex Polymorphism). *Lowercase names that are seemingly unbound are automatically quantified over in a prenex manner reminiscent of other functional languages like OCaml or Haskell. These variables are bound at quantity 0, meaning that they will be automatically erased during compilation.*

This function will interpret every **Desc** constructor as its obvious meaning as a type, using the parameter to give a meaning to **Rec** positions. In particular, **None** and **Prod** will respectively be interpreted by a unit type (**True**) and a product type (**Tuple**) defined below. We do not use the unit and product from the standard library purely to offer better syntactic sugar to our users.

```
record True where
  constructor I
```

```
record Tuple (a, b : Type) where
  constructor (#)
  fst : a
  snd : b
```

Idris Specifics 6 (Lack of Eta-rules for Records). **True** and **Tuple** are both defined as **records**. However in Idris 2, this is currently only syntactic sugar to declare a datatype together with a projection for each of the record's fields. Conversion checking does not incorporate eta rules for records and so we have to manually state, prove, and deploy eta rules whenever necessary. We include the two lemmas below as we will need them later.

```
etaTrue : (t : True) -> t === I
etaTrue I = Refl
```

```
etaTuple : (p : Tuple a b) -> p === (fst p # snd p)
etaTuple (t # u) = Refl
```

Now that we have these two type constructors, we can explicitly define **Meaning**, the function giving us the action on objects of the **Desc**-encoded endofunctors.

```
Meaning None x = True
Meaning Byte x = Bits8
Meaning (Prod d e) x = Tuple (Meaning d x) (Meaning e x)
Meaning Rec x = x
```

Both `None` and `Byte` are interpreted by constant functors (respectively the one returning the unit type `True`, and the one returning the type of bytes). `(Prod d e)` is interpreted as a tuple grouping the respective interpretations of `d` and `e`. Finally `Rec` is the identity functor.

Now that we have the action of descriptions on types, let us see their action on morphisms: provided a function from `a` to `b`, we can build one from the `Meaning` of `d` at type `a` to its meaning at type `b`. We once again proceed by induction on the description.

```
fmap : (d : Desc r s o) -> (a -> b) -> Meaning d a -> Meaning d b
fmap None f v = v
fmap Byte f v = v
fmap (Prod d e) f (v # w) = (fmap d f v # fmap e f w)
fmap Rec f v = f v
```

All cases but the one for `Rec` are structural. We leave out the proofs verifying that these definitions respect the functor laws up to pointwise equality. They are included in the supplementary material (File `Data.Serialisable.Data`).

3.2 Data as Trees

Given a datatype description `cs`, our first goal is to define what it means to pick a constructor. The `Index` record is a thin wrapper around a `(Fin (consNumber cs))` i.e. a finite natural number known to be smaller than the number of constructors the `cs` type provides.

```
record Index (cs : Data nm) where
  constructor MkIndex
  getIndex : Fin (consNumber cs)
```

We use this type rather than `Fin` directly because it plays well with inference. In the following code snippet, implementing a function returning the description corresponding to a given index, we use this to our advantage: the `cs` argument can be left implicit because it already shows up in the type of the `Index` and can thus be reconstructed by unification (Abel and Pientka, 2011).

```
description : {cs : Data nm} -> (k : Index cs) ->
  let cons = index (getIndex k) (constructors cs) in
  Desc True (static cons) (offsets cons)
description {cs} k
  = description (index (getIndex k) (constructors cs))
```

The `index` function is provided by the standard library: given a position of type `(Fin n)` and a vector of size `n`, it returns the value located at that position. Note that we seem to define `description` in terms of itself, in a manner that would create an infinite loop. But the occurrence on the right-hand side is actually referring to the projection out of the `Constructor` record. This is possible thanks to Idris' type-directed disambiguation.

Idris Specifics 7 (Type-directed Disambiguation). *If multiple definitions in scope have the same name, Idris performs type-directed disambiguation to pick the only one that would work in that context.*

This type of indices also allows us to provide users with syntactic sugar enabling them to use the constructors' names directly rather than confusing numeric indices. The following function runs a semi-decision procedure `isConstructor` (whose implementation is not given here) at the type level in order to turn any raw string `str` into the corresponding `Index`.

```
fromString : {cs : Data String} -> (str : String) ->
    {auto 0 _ : IsJust (isConstructor str cs)} ->
    Index cs
fromString {cs} str with (isConstructor str cs)
  _ | Just k = MkIndex k
```

Idris Specifics 8 (Ad-hoc Polymorphisms for Literals). *String, numeric, and floating point literals are respectively desugared using `fromString`, `fromInteger`, and `fromDouble`. Combined with the type-directed disambiguation (see above) of overloaded symbols, this allows users to compute potentially complex data from literals in a type-directed manner.*

In this instance, this allows us to use string literals as proxies for constructor names. If the string literal stands for a valid name then `isConstructor` will return a valid `Index` and the compiler's proof search mechanism will be able to automatically fill-in the implicit proof. If the name is not valid then Idris 2 will not find the index and will raise a compile time error. We include a successful example on the left and a failing test on the right hand side where the compiler is not able to find a proof of `(isJust (isConstructor "Cons" Tree))` because it is simply not the case that `"Cons"` is the name of a `Tree` constructor.

<pre>indexLeaf : Index Tree indexLeaf = "Leaf"</pre>	<pre>failing notIndexCons : Index Tree notIndexCons = "Cons"</pre>
--	--

Idris Specifics 9 (Failing Blocks). A **failing** block is a way to document (and enforce) that some code leads to an error. Such blocks are only accepted if their body parses but leads to an error during elaboration.

Once equipped with the ability to pick constructors, we can define the type of algebras for the functor described by a `Data` description. For each possible constructor, we demand an algebra for the functor corresponding to the meaning of the constructor's description.

```
Alg : Data nm -> Type -> Type
Alg cs x = (k : Index cs) -> Meaning (description k) x -> x
```

We can then introduce the fixpoint of data descriptions as the initial algebra, defined as the following inductive type.

```

553 data Mu : Data nm -> Type where
554   (#) : Alg cs (assert_total (Mu cs))

```

Idris Specifics 10 (Escape hatches). *Idris 2 provides some escape hatches to use when the author knows a usage is safe but the compiler is not able to determine that it is.*

A function call of the form (**assert_total** *e*) will circumvent the termination and positivity checkers in the expression *e*. This is only safe if the function is actually terminating or the type strictly positive.

A function call of the form (**believe_me** *e*) will be usable at any type. This is only safe if *e*'s actual type and the type it is being used at have the same runtime representation.

Note that here we are forced to use **assert_total** to convince Idris 2 to accept the definition. Indeed, unlike Agda, Idris 2 does not (yet!) track whether a function's arguments are used in a strictly positive manner. Consequently the positivity checker is unable to see that **Meaning** uses its second argument in a strictly positive manner, and so that **Alg** also is, and that this is therefore a legal definition.

Now that we can build trees as fixpoints of the meaning of descriptions, we can define convenient aliases for the **Tree** constructors. Note that the leftmost (#) use in each definition corresponds to the **Mu** constructor while later ones are **Tuple** constructors. Idris 2's type-directed disambiguation of constructors allows us to use this uniform notation for all of these pairing notions.

```

574 leaf : Mu Tree
575 leaf = "Leaf" # I
576
577 node : Mu Tree -> Bits8 -> Mu Tree -> Mu Tree
578 node l b r = "Node" # l # b # r

```

This enables us to define our running example as an inductive value of type (**Mu Tree**):

```

579 example : Mu Tree
580 example = (node (node (node leaf 1 leaf) 5 leaf) 10 (node leaf 20 leaf))

```

3.3 Generic Fold

Mu gives us the initial fixpoint for these algebras i.e. given any other algebra over a type **a**, from a term of type (**Mu cs**), we can compute an **a**. We define the generic **fold** function over inductive values as follows:

```

587 fold : {cs : Data nm} -> Alg cs a -> Mu cs -> a
588 fold alg (k # t) = alg k (assert_total $ fmap _ (fold alg) t)

```

We first match on the term's top constructor, use **fmap** (defined in [Section 3.1](#)) to recursively apply the fold to all the node's subterms and finally apply the algebra to the result. Here we only use **assert_total** because Idris 2 does not see that **fmap** only applies its argument to strict subterms. This limitation could easily be bypassed by mutually defining an inlined and specialised version of (**fmap _ (fold alg)**) as we demonstrate in [Appendix 1](#). In an ideal type theory these supercompilation steps, whose sole purpose is to satisfy the totality checker, would be automatically performed by the compiler (Mendel-Gleason, 2012).

Further generic programming can yield other useful programs e.g. a generic proof that tree equality is decidable, a generic definition of zippers (Löh and Magalhães, 2011), or a tail-recursive version of fold (Tomé Cortiñas and Swierstra, 2018).

4 Serialised Representation

Before we can give a meaning to descriptions as pointers into a buffer we need to decide on a serialisation format. The format we have opted for is split in two parts: a header, followed by the actual representation of the tree as a contiguous block of bytes. The header contains an encoding of the description of the type of the tree stored in the rest of the buffer. It can be used when loading a file to check that a user’s claim about the presence of a serialised tree of a given type is correct.

For instance, the following binary snippet is a hex dump of a file containing the serialised representation of a binary tree belonging to the type we have been using as our running example. The raw data is semantically highlighted: 8-bytes-long¹ offsets in little endian format, a **type** description of the stored data, some **nodes** of the tree and the **data** stored in the nodes.

```

87654321 00 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF
00000000: 07 00 00 00 00 00 00 00 02 00 02 03 02 01 03 01
00000010: 17 00 00 00 00 00 00 00 01 0c 00 00 00 00 00 00
00000020: 00 01 01 00 00 00 00 00 00 00 00 01 00 05 00 0a
00000030: 01 01 00 00 00 00 00 00 00 00 00 14 00

```

More specifically, this block is the encoding of the **example** given in the previous section and, accepting that the tree’s representation starts at byte FF, and knowing that a **leaf** is represented here by 00 and a **node** is represented by 01 readers can check (ignoring the offsets for now) that the data is stored in a depth-first, left-to-right traversal of the tree i.e. we get exactly the bit pattern we saw in the naïve encoding presented in [Section 1](#):

```

      (node (node leaf 1 leaf) 5 leaf)
      └──────────────────────────┘
01 ... 01 ... 01 ... 00 01 00 05 00 0a 01 ... 00 14 00
      └──────────────────────────┘
      (node leaf 1 leaf)

```

Let us now look at the format more closely.

4.1 Header

In our example, the header is as follows:

```
07 00 00 00 00 00 00 00 02 00 02 03 02 01 03
```

¹ This implicitly means that we assume that our terms occupy less than 2⁶⁴ bytes, a very reasonable assumption in our opinion.

The header consists of an 8-bytes long offset stored in little endian followed by a binary representation of the [Data](#) description of the value stored in the buffer. The offset allows us to jump past the description in case we do not care to inspect it.

This description can be useful in a big project where different components produce and consume such serialised values: if we change the format in one place but forget to update it in another, we want the program to gracefully fail to load the file using an unexpected format. We detail in [Section 11.2.2](#) how dependent type providers (Christiansen, 2013) can help structure a software project to prevent such issues at compile time.

The encoding of a data description starts with a byte giving us the number of constructors², followed by these constructors' respective descriptions serialised one after the other. **None** is represented by **00**, **Byte** is represented by **01**, (**Prod** **d** **e**) is represented by **02** followed by the representation of **d** and then that of **e**, and **Rec** is represented by **03**.

Looking once more at the header in the running example, the `Data` description is indeed 7 bytes long like the offset states. The `Data` description starts with `02` meaning that the type has two constructors. The first one is `00` i.e. `None` (this is the encoding of the type of `Leaf`), and the second one is `02 03 02 01 03` i.e. `(Prod Rec (Prod Byte Rec))` (that is to say the encoding of the type of `Node`). According to the header, this file does contain a `Tree`.

4.2 Tree Serialisation

The design of this format was guided by a few simple principles detailed in [Section 2.1](#):

1. the format should be contiguous (no pointer indirections)
2. values of type unit should occupy no space
3. the nesting of pairs in a description should have no impact on the layout
4. the format should support the direct access to any of a node's subtrees

This last criterion allows us to skip past subtrees that we do not need to process thus ensuring an exponential speedup compared to formats forcing an in-order traversal of a node’s subtrees. To this end each node needs to store an offset measuring the size of the subtrees that are to the left of any relevant information.

If a given tag is associated to a description of type (`Desc True s o`) then the representation in memory of the associated node will look something like the following. On the first line we have a description of the data layout and on the second line we have the offset of various positions in the block with respect to the tag's address.

tag	o offsets	tree ₁ ⋯ byte ₁ ⋯ tree _{k} ⋯ byte _{s} tree _{$o+1$}	
0	1	$1 + 8 \times o$	$8 \times o + s + \sum_{i=1}^o o_i$

For the data layout, we start with the tag then we have o offsets, and finally we have a block contiguously storing an interleaving of subtrees and s bytes dictated by the description.

² Remember that we enforce in the definition of the `Data` record type in [Section 2.4](#) that our descriptions cannot have more than 255 constructors. So using a single byte here is safe.

In this example the rightmost value in the description is a subtree and so even though we have o offsets, we actually have $(o + 1)$ subtrees stored.

The offsets of the tag with respect to its own address is 0. The tag occupies one byte and so the offset of the block of offsets is 1. Each offset occupies 8 bytes and so the constructor's arguments are stored at offset $(1 + 8 \times o)$. Finally each value's offset can be computed by adding up the offset of the start of the block of constructor arguments, the offsets corresponding to all of the subtrees that come before it, and the number of bytes stored before it; in the case of the last byte that gives $1 + 8 \times o + \sum_{i=1}^o o_i + s - 1$ hence the formula included in the diagram.

Going back to our running example, this translates to the following respective data layouts and offsets for a leaf and a node.

Leaf	Node				
00	01	offset	left subtree	byte	right subtree
0	0	1	9	$9 + o_1$	$10 + o_1$

We now have a good understanding of the serialisation format we are going to use to represent our inductive trees. The next step is to define what it means to have a pointer to a tree residing in a buffer.

5 Interlude: Logics for Imperative Programs

Let us first look back at some logical frameworks built to explicitly talk about memory locations and their contents in order to prove the properties of imperative programs. Their design and properties will then inform our choices.

5.1 Hoare Logic

Hoare logic (1969) is a framework in which we can state and prove the properties of imperative programs. Its central notion is that of Hoare triples. They are statements of the form

$$\{ P \} e \{ v. Q \}$$

declaring that under the precondition P , and binding the result of evaluating the expression e as v , we can prove that Q holds.

The most basic of predicates to express knowledge about the memory state is a 'points to' assertion $(\ell \mapsto w)$ stating that the label ℓ points to a memory location containing the word w .

This can be used for instance to specify the behaviour of a primitive dereferencing operator: if ℓ is known to point to a value w then evaluating the program $(\text{deref}(\ell))$ will return a value v equal to w .

$$\{ \ell \mapsto w \} \text{deref}(\ell) \{ v. v = w \}$$

Provided that we only have read-only operations, evaluating a program cannot invalidate assumptions about the contents of the memory and so we enjoy a form of weakening: for any proposition R , if we can specify the behaviour of a program e using the precondition P and the postcondition Q , then the same program will also abide by the specification using the precondition $(P \wedge R)$ and the postcondition $(Q \wedge R)$.

$$\{ P \} e \{ v. Q \} \implies \{ P \wedge R \} e \{ v. Q \wedge R \}$$

In particular, the validity of $(\ell \mapsto w)$ can be propagated even after having dereferenced it.

Using further structural rules, we can combine the axioms to compositionally prove statements about more complex programs. We present below a structural rule for a `let` construct: provided that executing e returns a value v satisfying Q under precondition P and that for all x , executing e' returns a value v satisfying R under precondition $Q(x)$ then executing $(\text{let } x = e \text{ in } e')$ will return a value v satisfying R under precondition P .

$$\begin{aligned} \{ P \} e \{ v. Q \} \wedge \forall x. \{ Q(x) \} e' \{ v. R \} \\ \implies \{ P \} \text{let } x = e \text{ in } e' \{ v. R \} \end{aligned}$$

We can for instance prove that we can chase pointers: if ℓ points to a value ℓ' itself corresponding to a label pointing to w , nested calls to `deref` will return a value equal to w :

$$\{ \ell \mapsto \ell' \wedge \ell' \mapsto w \} \text{let } x = \text{deref}(\ell) \text{ in } \text{deref}(x) \{ v. v = w \}$$

Once we have these building blocks, it becomes interesting to introduce some abstractions. As we explained above, $(\ell \mapsto w)$ states that the memory location called ℓ contains the word w . The programs we are interested in do not however manipulate isolated words, they talk about full blown inductive trees. The key idea is to use this base ‘points to’ assertion as a building block to compositionally give a meaning to richly typed pointers. We will write $(\ell \xrightarrow{ty} v)$ for the assumption that the label ℓ points to a value v of type ty . We can naturally define $(\ell \xrightarrow{\text{Bits8}} w)$ as simply $(\ell \mapsto w)$. We can then decide that, assuming that we know the size s_a of values of type a , $(\ell \xrightarrow{(a, b)} (v_1, v_2))$ means that ℓ points to two contiguous values v_1 and v_2 . In other words it is an alias for $(\ell \xrightarrow{a} v_1 \wedge \ell + s_a \xrightarrow{b} v_2)$.

These richly typed pointers are the kind of pointers we are going to define in the next section. But before, let us have a look at how things get more complicated once we are given access to primitives that can destructively update the contents of a memory location.

5.2 Separation Logic

It is tempting to add a new primitive `assign` that takes a label ℓ and a word w and updates the memory cell so that $(\ell \mapsto w)$ now holds true. Its specification would be

$$\{ \ell \mapsto _ \} \text{assign}(\ell, w) \{ _ . \ell \mapsto w \}$$

However with such a primitive things that were true before a program was run may have been invalidated in the process. Consequently the weakening principle given above does not hold anymore. Otherwise the following reasoning step would be valid and the memory cell

corresponding to the label ℓ would need to contain both 0 and 1.

$$\begin{aligned} & \{ \ell \mapsto 0 \} \text{ assign } (\ell, 1) \{ _ . \ell \mapsto 1 \} \\ \not\Rightarrow & \{ \ell \mapsto 0 \wedge \ell \mapsto 0 \} \text{ assign } (\ell, 1) \{ _ . \ell \mapsto 1 \wedge \ell \mapsto 0 \} \end{aligned}$$

This makes Hoare logic highly non-compositional in the presence of destructive updates: having proven a program fragment’s specification using predicates mentioning the memory locations it interacts with, this result cannot be reused when proving the specification of a larger program involving additional memory locations.

The state-of-the-art solution is to move away from Hoare logic and use separation logic (Reynolds, 2002; O’Hearn, 2019; Charguéraud, 2023,?) whose core idea is to have a *separating* conjunction: $(P * Q)$ states that both P and Q hold true but that they are talking about *separate* slices of the memory. This allows for the safe inclusion of a similar weakening principle, the *frame rule* which states that for an arbitrary predicate R if e can be specified using precondition P and postcondition Q then it also can for precondition $(P * R)$ and postcondition $(Q * R)$ precisely because R is only talking about memory locations not impacted by e .

As Rouvoet (2021) demonstrated, it is possible to embed separation logic in type theory but, even with best efforts, it remains a somewhat heavy process. Luckily for us, we do not need to.

Our goal with this work is to process data serialised in buffers without first deserialising it as an in-memory tree. To do so we only need to be able to read from the buffer to analyse the structure of the inductive data stored in it. Although we may *want* to have destructive updates in future work, for this specific task we do not actually *need* them. Correspondingly, we will happily stick to a Hoare-style logic and its really powerful weakening principle. As a matter of fact, we will go even further and entirely bypass the usual explicit embedding of such logics in the host language. We will use Idris 2’s abstraction facilities to introduce richly typed ‘points to’ predicates as first class values that can just be passed around, and trusted primitives that can take these pointers and reveal the shape of the values they are pointing to.

6 Meaning as Pointers Into a Buffer

For reasons that will become apparent in Section 8.5 when we start programming over serialised data in a correct-by-construction manner, our types of ‘pointers’ will be parameterised not only by the description of the type of the data stored but also by a runtime-irrelevant inductive value of that type.

Note that we will *not* export the constructors to the various ‘pointer’ types defined in the following section. Consequently, the only way for a user to get their hands on such a pointer is to use the library functions we provide. By only providing invariant-respecting functions, we can ensure that the assumptions encoded in the phantom parameters are never violated.

6.1 Tracking Buffer Positions

We start with the definition of `Pointer.Mu`, the counterpart to `Data.Mu` for serialised values.

```

829 record Mu (cs : Data nm) (t : Data.Mu cs) where
830   constructor MkMu
831   muBuffer : Buffer
832   muPosition : Int
833   muSize : Int

```

A tree sitting in a buffer is represented by a record packing the buffer, the position at which the tree's root node is stored, and the size of the tree. The record is indexed by `cs` a `Data` description and `t` the tree of type `(Data.Mu cs)` which is represented by the buffer's content. Neither are mentioned in the types of the record's fields, making them *phantom types* (Leijen and Meijer, 1999). A term of type `(Pointer.Mu cs t)` plays a double role in our library: it acts both as a label ℓ in the runtime relevant layer, and a proof that said label points to a buffer-bound value equal to t (i.e. $\ell \xrightarrow{\text{Data.Mu cs}} t$).

The pointer counterpart to a `Meaning` stores additional information.

```

843 record Meaning (d : Desc r s o) (cs : Data nm)
844   (t : Data.Meaning d (Data.Mu cs)) where
845   constructor MkMeaning
846   subterms : Vect o Int
847   meaningBuffer : Buffer
848   meaningPosition : Int
849   meaningSize : Int

```

For a description of type `(Desc r s o)` on top of the buffer, the position at which the root of the meaning resides, and the size of the layer we additionally have a vector of `o` offsets corresponding to the sizes of the subtrees we may want to skip past. This will allow us to efficiently access any constructor argument we want.

6.2 Writing a Tree to a File

Once we have a pointer to a tree `t` of type `cs` (`Pointer.Mu cs t` in the type below) in a buffer, we can easily write it to a file be it for safekeeping or sending over the network.

```

861 writeToFile : {cs : Data nm} -> FilePath ->
862   forall t. Pointer.Mu cs t -> IO ()
863 writeToFile fp (MkMu buf pos size) = do
864   desc <- getInt buf 0
865   let start = 8 + desc
866   let bufSize = 8 + desc + size
867   buf <- if pos == start then pure buf else do
868     Just newbuf <- newBuffer bufSize
869     | Nothing => failWith "{__LOC__} Couldn't allocate buffer"
870   copyData buf 0 start newbuf 0
871   copyData buf pos size newbuf start
872   pure buf
873 Right () <- writeBufferToFile fp buf bufSize
874 | Left (err, _) => failWith (show err)
875 pure ()

```

Idris Specifics 11 (Forall Quantifier). *The **forall** quantifier is sugar for an implicit binder at quantity 0. It can be useful to introduce variables that cannot be automatically bound in a prenex manner because they have a type dependency over an explicitly bound argument.*

We first start by reading the size of the header stored in the buffer. This allows us to compute both the **start** of the data block as well as the size of the buffer (**bufSize**) that will contain the header followed by the tree we want to write to a file. We then check whether the position of the pointer is exactly the beginning of the data block. If it is then we are pointing to the whole tree and the current buffer can be written to a file as is. Otherwise we are pointing to a subtree and need to separate it from its surrounding context first. To do so we allocate a new buffer of the right size and use the standard library's **copyData** primitive to copy the raw bytes corresponding to the header first, and the tree of interest second. We can then write the buffer we have picked to a file and happily succeed.

6.3 Reading a Tree from a File

We can also go in the other direction: using the data description **cs**, we can load the content of the file located at **fp** as a pointer to the root of the tree of type (**Mu cs**) we claim is contained in it.

```
readFromFile : {default True safe : Bool} ->
  (cs : Data nm) -> FilePath -> IO (Exists (Pointer.Mu cs))
readFromFile cs fp
  = do Right buf <- createBufferFromFile fp
      | Left err => failWith (show err)
  skip <- getInt buf 0
  when safe $ do
    cs' <- getData buf 8
    unless (eqData cs cs') $ failWith $ unlines
      [ "Description mismatch:"
        , "expected:", show cs, "but got:", show cs' ]
  let pos = skip + 8
  pure (Evidence t (MkMu buf pos (!(rawSize buf) - pos)))

where 0 t : Data.Mu cs -- postulated as an abstract value
```

This function takes a default argument **safe** controlling whether we should attempt to check that the file starts with a header containing a type description matching the one passed as an argument by the caller.

Idris Specifics 12 (Default Arguments). *An implicit argument can be assigned a **default** value. It will take this value unless explicitly overwritten by the caller.*

First, we create a buffer from the content of the file. We then read the offset giving us the size of the header. If we want to be **safe** we then read the type description contained in the header using **getData** (not shown here) and check it for equality against the one we were passed using **eqData** (not shown here). If the check fails, we emit an error and fail. If the

check succeeds, we postulate the existence of a runtime irrelevant tree meant to represent the file’s content and put together a pointer for that abstract tree. This postulate cements the user’s claim about the file’s content; naturally if the file is not in fact contain a serialised representation of a tree this can lead to fatal errors later on when attempting to inspect the buffer’s content. We return a pair of the runtime irrelevant tree and a pointer to it thus ensuring users cannot directly attempt to match on the inductive tree; they will have to use the combinators we are about to define to inspect it by reading into the buffer.

Note that, in order to save space in the paper, we never checked whether the buffer reads we performed in both `writeToFile` and `readFromFile` were within bounds. A released version of the library would naturally need to include such checks.

Now that we have pointers, and use files to read and write the trees they are standing for, we are only missing the ability to look at the content they are pointing to. But first we need to introduce some basic tools to be able to talk precisely about this stored content.

7 Interlude: Views and Singletons

The precise indexing of pointers by a runtime-irrelevant copy of the value they are pointing to means that inspecting the buffer’s content should not only return runtime information but also refine the index to reflect that information at the type-level. As a consequence, the buffer-inspecting functions we are going to define will be views.

7.1 Views

A view in the sense of Wadler (Wadler, 1987), and subsequently refined by McBride and McKinna (McBride and McKinna, 2004) for a type T is a type family V indexed by T together with a function which maps values t of type T to values of type $V\ t$. By inspecting the $V\ t$ values we can learn something about the t input. The prototypical example is perhaps the ‘snoc’ (‘cons’ backwards) view of right-nested lists as if they were left-nested. We present the `Snoc` family below.

```
data Snoc : List a -> Type where
  Lin : Snoc []
  (:<) : (init : List a) -> (last : a) -> Snoc (init ++ [last])
```

By matching on a value of type `(Snoc xs)` we get to learn either that `xs` is empty (`Lin`, nil backwards) or that it has an initial segment `init` and a last element `last` (`init :< last`). Crucially `(init ++ [last])` is not a valid pattern because it mentions a stuck function call but `(init :< last)` is as it is only made up of constructors and binding positions. And so by having a function that computes the `(Snoc xs)` view of any list `xs`, we are able to pretend as if we were actually able to match on “patterns” of the form `(init ++ [last])`. The function `unsnoc` demonstrates that we can always view a `List` in such a `Snoc`-manner.

```

unsnoc : (xs : List a) -> Snoc xs
unsnoc [] = [<]
unsnoc (x :: xs@_) with (unsnoc xs)
  _ | [<] = [] :< x
  _ | init :< last = (x :: init) :< last

```

Idris Specifics 13 (With-Abstraction). The **with** construct allows programmers to locally define an anonymous auxiliary function taking an extra argument compared to its parent. By writing (**with** (*e*)) we introduce such an auxiliary function and immediately apply it to *e*. The nested clauses that immediately follow each take an extra pattern which matches over the possible values of *e*. If the left-hand side of the auxiliary function is the same as that of its parents bar the pattern for the newly added argument, we can use the elision notation (*_* |) to avoid having to repeat ourselves.

In other words the following definition of *f* using a **with** construct with the elision notation

```

f x y with (e)
  _ | p = rhs1
  _ | q = rhs2

```

is equivalent to the following desugared version where the auxiliary function *fAux* taking an extra argument has been lifted to the toplevel.

```

fAux x y p = rhs1
fAux x y q = rhs2

f x y = fAux x y e

```

In the code snippet for *unsnoc* we performed a recursive call on (*unsnoc xs*) and distinguished two cases: when the view returns the empty snoclist [<] and when it returns an *initial* segment together with the *last* element. Because we are using a view, matching on these constructors actually refined the shape of the parent clause's argument *xs*. We do not need to spell out its exact shape in each branch because we were careful to only introduce *xs* as a name alias using an as-pattern while letting the actual pattern be a catch-all pattern (*xs@_*). This is a common trick to make working with views as lightweight as possible.

Here we defined *Snoc* as an inductive family but it can sometimes be convenient to define the family recursively instead, in which case the *Singleton* inductive family can help us connect runtime values to their runtime-irrelevant type-level counterparts.

7.2 The Singleton Type

The *Singleton* family has a single constructor which takes an argument *x* of type *a*, its return type is indexed precisely by this *x*.

```

data Singleton : a -> Type where
  MkSingleton : (x : a) -> Singleton x

```

More concretely this means that a value of type (`Singleton t`) has to be a runtime relevant copy of the term t . Note that Idris 2 performs an optimisation similar to Haskell's `newtype` unwrapping: every datatype that has a single non-recursive constructor with only one non-erased argument is unwrapped during compilation. This means that at runtime the `Singleton` / `MkSingleton` indirections will have disappeared.

We can define some convenient combinators to manipulate singletons. We reuse the naming conventions typical of applicative functors which will allow us to rely on Idris 2's automatic desugaring of *idiom brackets* (McBride and Paterson, 2008) into expressions using these combinators.

First `pure` is a simple alias for `MkSingleton`, it turns a runtime-relevant value x into a singleton for this value.

```
pure : (x : a) -> Singleton x
pure = MkSingleton
```

Next, we can ‘map’ a function under a `Singleton` layer: given a pure function f and a runtime copy of t we can get a runtime copy of $(f\ t)$.

```
(<$>) : (f : a -> b) -> Singleton t -> Singleton (f t)
f <$> MkSingleton t = MkSingleton (f t)
```

Finally, we can apply a runtime copy of a function f to a runtime copy of an argument t to get a runtime copy of the result $(f\ t)$.

```
(<*>) : Singleton f -> Singleton t -> Singleton (f t)
MkSingleton f <*> MkSingleton t = MkSingleton (f t)
```

As we mentioned earlier, Idris 2 automatically desugars idiom brackets using these combinators.

Idris Specifics 14 (Idiom Brackets as Sugar). *Idiom brackets let us use the standard whitespace-based application typical of pure functions to perform computations in an Applicative contexts. In other words, during elaboration an atomic expression `[| x |]` will be desugared to `(pure x)` while a compound expression `[| f t1 ··· tn |]` will become `(f <$> t1 <*> ··· <*> tn)`.*

This built-in handling of idiom brackets lets us apply `Singleton`-wrapped values almost as seamlessly as pure values.

We are now equipped with the appropriate notions and definitions to look at a buffer's content.

8 Inspecting a Buffer's Content

Let us now describe the combinators allowing our users to take apart the values they have a pointer for. These functions will read bytes in the buffer and reflect the observations thus made at the type level by refining the pointer's indices.

There will be two separate tiers of definitions: the most basic building blocks (**poke** and **out**) will be a trusted core of primitives implemented using escape hatches. This is inevitable given that we are reflecting the content of buffer reads at the type level. We will clearly specify their behaviour by explaining what benign Hoare-style axioms they correspond to.

We will then show how we can use these low-level trusted primitives to define higher level combinators (**layer** and **view**). Crucially these definitions will not need to use further escape hatches: provided that the trusted core is correct, then so will they. This will culminate in the implementation of a generic correct-by-construction version of **fold** operating over trees stored in a buffer (cf. [Section 8.5](#)).

8.1 Poking the Buffer

Our most basic operation consists in poking the buffer to gain access to the head constructor of the underlying layer of **Data.Meaning** we have a pointer to. This operation is description-directed and so its result (called **Poke**) is defined by case analysis on the description associated to the pointer.

Concretely, the type of the function is as follows: provided a pointer for a description **d**, subtrees of type **cs** and an associated meaning **t** of type (**Meaning cs t**) we return an **IO** process computing the one step unfolding of the meaning.

```
poke : {0 cs : Data nm} -> {d : Desc r s o} ->
      forall t. Pointer.Meaning d cs t ->
      IO (Poke d cs t)
```

As we explained, **Poke** is defined by case-analysis on the description. However, in order to keep the notations user-friendly, we are forced by Idris' lack of eta-rules (cf. [Remark 6](#)) to mutually define an inductive family **Poke'** with interesting return indices. It will allow users to, by matching on **Poke'** constructors, automatically refine the associated meaning present at the type level into a term with a head constructor. This will ensure that functions defined by pattern-matching can reduce in types based on observations made at the term level.

```
Poke : (d : Desc r s o) -> (cs : Data nm) ->
      Data.Meaning d (Data.Mu cs) -> Type
Poke Byte cs t = Singleton t
Poke Rec cs t = Pointer.Mu cs t
Poke d cs t = Poke' d cs t
```

Poking a buffer containing a **Byte** will yield a runtime-relevant copy of the type-level byte we have for reference, hence the use of **Singleton**. This corresponds to adding the following Hoare-style axiom for **poke**. Remembering that (**Meaning Byte (Mu cs)**) computes to **Bits8** and so that ($\ell \xrightarrow{\text{Meaning Byte (Mu cs)}} w$) is essentially ($\ell \mapsto w$), we note that this

Hoare-style axiom looks eerily similar to the axiom for `deref` we gave in [Section 5.1](#)):

$$\{ \ell \vdash \text{Meaning Byte (Mu cs)} \rightarrow w \} \text{poke } \ell \{ v. v = w \}$$

If the description is `Rec` this means we have a substructure. In this case we simply demand a pointer to it. This amounts to adding the following axiom:

$$\{ \ell \vdash \text{Meaning Rec (Mu cs)} \rightarrow t \} \text{poke } \ell \{ \ell'. \ell' \vdash \text{Mu cs} \rightarrow t \}$$

Last but not least, if we are accessing a value of a record type (either `None` or a `Prod` of two descriptions) then we describe the resulting observation using the `Poke'` family.

```
data Poke' : (d : Desc r s o) -> (cs : Data nm) ->
  Data.Meaning d (Data.Mu cs) -> Type where
  I : Poke' None cs I
  (#) : Pointer.Meaning d cs t ->
    Pointer.Meaning e cs u ->
    Poke' (Prod d e) cs (t # u)
```

`Poke'` has two constructors corresponding to the two descriptions it covers. If the description of the buffer's content is `None` then we do not expect to get a value back, only the knowledge that the type-level meaning is `I`. This corresponds to adding the following axiom.

$$\{ \ell \vdash \text{Meaning None (Mu cs)} \rightarrow t \} \text{poke } \ell \{ _ . t = I \}$$

If the description is `(Prod d e)` then we demand to learn that the type-level term is `(#)`-headed with two substructures t_1 and t_2 and we expect `poke` to give us a pointer to each of these substructures. This corresponds to the following axiom.

$$\{ \ell \vdash \text{Meaning (Prod d e) (Mu cs)} \rightarrow t \} \\ \text{poke } \ell \\ \{ (\ell_1, \ell_2). \exists t_1. \exists t_2. t = (t_1 \# t_2) \wedge \ell_1 \vdash \text{Meaning d (Mu cs)} \rightarrow t_1 \wedge \ell_2 \vdash \text{Meaning e (Mu cs)} \rightarrow t_2 \}$$

As we mentioned earlier, the Idris 2 implementation of the `poke` function is necessarily using escape hatches as we are essentially giving a computational content to the axioms listed above. We proceed by case analysis on the description. Let us go through each case one-by-one.

```
poke {d = None} el = pure (rewrite etaTrue t in I)
```

If the description is `None` we do not need to fetch any information from the buffer but we do need to deploy the eta rule for `True` (cf. [Remark 6](#) for the definition of `etaTrue`) in order to be able to use the `Poke'` constructor `I`.

```
poke {d = Byte} el = do
  bs <- getBits8 (meaningBuffer el) (meaningPosition el)
  pure (unsafeMkSingleton bs)
```

If the description is `Byte` then we read a byte at the determined position. The only way we can connect this value we just read to the runtime irrelevant type index is to use the unsafe combinator `unsafeMkSingleton` to manufacture a value of type `(Singleton t)` instead of the value of type `(Singleton bs)` we would expect from wrapping `bs` in the `MkSingleton` constructor. As we explained earlier, this amounts to realising the Hoare-style axiom specifying the act of dereferencing a pointer.

```
poke {d = Prod {s1, ol} d e} {t} (MkMeaning sub buf pos size) = do
  let (sub1, subr) = splitAt ol sub
  let sizel = sum sub1 + cast s1
  let left = MkMeaning sub1 buf pos sizel
  let posr = pos + sizel
  let right = MkMeaning subr buf posr (size - sizel)
  pure (rewrite etaTuple t in left # right)
```

If the description is the product of two sub-descriptions then we want to compute the `Pointer.Meaning` corresponding to each of them. We do so by following the serialisation format we detailed in Section 4. We start by splitting the vector of offsets to distribute them between the left and right subtrees.

We can readily build the pointer for the `left` subdescription: it takes the left offsets, the buffer, and has the same starting position as the whole description of the product as the submeanings are stored one after the other. Its size (called `sizel`) is the sum of the space reserved by all of the left offsets (`sum sub1`) as well as the static size occupied by the rest of the content (`s1`).

We then compute the starting position of the right subdescription: we need to move past the whole of the left subdescription, that is to say that the starting position is the sum of the starting position for the whole product and `sizel`. The size of the right subdescription is then easily computed by subtracting `sizel` from the overall `size` of the paired subdescriptions.

We can finally use the lemma `etaTuple` (defined in Remark 6) saying that a tuple is equal to the pairing of its respective projections in order to turn `t` into `(fst t # snd t)` which lets us use the `Poke'` constructor `(#)` to return our pair of pointers.

Although we did not need to use escape hatches here, the implementation is still part of the trusted core in that we are computing offsets in (we claim!) accordance with the serialisation format.

```
poke {d = Rec} (MkMeaning _ buf pos size) = pure (MkMu buf pos size)
```

Lastly, when we reach a `Rec` description, we can discard the vector of offsets and return a `Pointer.Mu` with the same buffer, starting position and size as our input pointer.

8.2 Extracting One Layer

By repeatedly poking the buffer, we can unfold a full layer of term. This operation is not part of the trusted core: provided that `poke` is correct then it will automatically be correct-by-construction. The result type is once again defined by induction on the description.

It is essentially identical to the definition of `Poke` except for the `Prod` case: instead of being content with a pointer for each of the subdescriptions, we demand a `Layer` for them too.

```

Layer : (d : Desc r s o) -> (cs : Data nm) ->
        Data.Meaning d (Data.Mu cs) -> Type
Layer Byte _ t = Singleton t
Layer Rec cs t = Pointer.Mu cs t
Layer d cs t = Layer' d cs t

data Layer' : (d : Desc r s o) -> (cs : Data nm) ->
        Data.Meaning d (Data.Mu cs) -> Type where
  I : Layer' None cs I
  (#) : Layer d cs t -> Layer e cs u -> Layer' (Prod d e) cs (t # u)

```

This function can easily be implemented by induction on the description and repeatedly calling `poke` to expose the values one by one. We call `poke` and use the `IO` monad's bind operator (`>>=`) to pass the result to `go`, the auxiliary function recursively going under record constructors to perform further poking.

```

layer : {0 cs : Data nm} -> {d : Desc r s o} ->
        forall t. Pointer.Meaning d cs t -> IO (Layer d cs t)
layer el = poke el >>= go d where

  go : forall r, s, o. (d : Desc r s o) ->
        forall t. Poke d cs t -> IO (Layer d cs t)
  go None I = pure I
  go Byte p = pure p
  go (Prod d e) (p # q) = [| layer p # layer q |]
  go Rec p = pure p

```

If the description is `None`, `Byte`, or `Rec` then poking the buffer was already enough to reveal the full layer and we can simply return the result. If we have a `(Prod d e)` then poking the buffer revealed that the term is `(#)`-headed and handed us a pointer to each of its components. We call `layer` recursively on each of these pointers and use idiom brackets to combine the `IO`-wrapped results using `Layer'`'s `(#)` constructor.

We can readily use this function to inspect meanings we have a pointer to. In the following artificial example, looking at the goal `?hole`, we learn that `v` has the shape `(t # (I # u))` where `t` is a tree of type `(Mu cs)` and `u` is a byte, and that `q` is a pointer to `t` and `w` has type `(Singleton u)`.

```

example : Pointer.Meaning (Prod Rec (Prod None Byte)) cs v -> IO ()
example p = layer p >>= \ (q # I # w) => ?hole

```

We have effectively managed to take `v` apart in a type-directed manner and get a handle on its subterms. In that sense we have just defined a view for meanings stored in buffers!

8.3 Exposing a Tree's Top Node

Now that we can deserialise an entire layer of `Meaning`, the only thing we are missing to be able to generically manipulate trees is the ability to expose the top node of a tree stored at a `Pointer.Mu` position. This will require the addition of a new function to the trusted core: the function `out`. Its type states that given a pointer to a tree `t` of type `cs` we can get an `IO` process revealing the top node of `t`.

```
out : {cs : Data nm} -> forall t. Pointer.Mu cs t -> IO (Out cs t)
```

The `Out` family formally describes what revealing the top node means: obtaining an `Index` named `k`, and a `Pointer.Meaning` to the meaning `t` of the description associated to `k`. The family's index `(k # t)` (where the overloaded `(#)` is here the `Data.Mu` constructor) ensures that the structure of the runtime irrelevant tree is adequately described by this index and meaning.

```
data Out : (cs : Data nm) -> (t : Data.Mu cs) -> Type where
  (#) : (k : Index cs) ->
    forall t. Pointer.Meaning (description k) cs t ->
      Out cs (k # t)
```

This amounts to introducing the following Hoare-style axiom for `out`:

$$\{ \ell \xrightarrow{\text{Mu cs}} t \} \text{out } \ell \{ (k, \ell_1). \exists t_1. t = (k \# t_1) \wedge \ell_1 \xrightarrow{\text{Meaning cs}_k (\text{Mu cs})} t_1 \}$$

The function `out` being part of the trusted core means that its implementation will once again need to use escape hatches to reconcile the buffer's observed content with the type-level indices. Let us recall the data layout detailed in [Section 4.2](#):

tag	<i>o</i> offsets	tree ₁ ⋯ byte ₁ ⋯ tree _{<i>k</i>} ⋯ byte _{<i>s</i>} tree _{<i>o</i>+1}
0	1	1 + 8 × <i>o</i> 8 × <i>o</i> + <i>s</i> + ∑ _{<i>i</i>=1} ^{<i>o</i>} <i>o_i</i>

Operationally, `out` will amount to inspecting the tag used by the node, deserialising the offsets stored immediately after it, and forming a pointer to the start of the meaning block. As a first step, let us get our hands on the index of the head constructor.

```
getIndex : {cs : Data nm} -> forall t. Pointer.Mu cs t -> IO (Index cs)
getIndex mu = do
  tag <- getBits8 (muBuffer mu) (muPosition mu)
  let Just k = natToFin (cast tag) (consNumber cs)
      | _ => failWith "Invalid representation"
  pure (MkIndex k)
```

We obtain a byte by calling `getBits8`, cast it to a natural number and then make sure that it is in the range `[0 ··· consNumber cs]` using `naturToFin`. If the check fails then we return a

hard fail: the buffer contains an invalid representation and so the precondition that we only have pointers to valid values was violated.

We can now describe the `out` function's implementation.

```
out {t} mu = do
  k <- getIndex mu
  let 0 sub = unfoldAs k t
  val <- (k #) <$> getConstructor k {t = sub.fst}
                (rewrite sym sub.snd in mu)
  pure (rewrite sub.snd in val)
```

We start by reading the index `k` corresponding to the constructor choice. We then use the unsafe `unfoldAs` postulate to step the type-level `t` to something of the form `(k # val)`. We then conclude using the `getConstructor` function (defined later) to gather the required offsets and put together the pointer to the meaning of the description associated to `k`.

```
%unsafe
0 unfoldAs :
  (k : Index cs) -> (0 t : Data.Mu cs) ->
  (val : Data.Meaning (description k) (Data.Mu cs)
   ** t === (k # val))
```

The declaration of `unfoldAs` is marked as runtime irrelevant because it cannot possibly be implemented (`t` is runtime irrelevant and so cannot be inspected) and so its output should not be relied upon in runtime-relevant computations. Its type states that there exists a `Meaning` called `val` such that `t` is equal to `(k # val)`. This is of course untrue in general: we cannot take an arbitrary `t` and declare that it is `k`-headed. This use-case is however justified in that it reflects at the type-level the observation we made by reading the buffer.

Now that we know the head constructor we want to deserialise and that we have the ability to step the runtime irrelevant tree to match the actual content of the buffer, we can use `getConstructor` to build such a value. Given a pointer to a tree `(k # t)`, it will read enough information from the buffer to assemble a pointer to the meaning `t`.

```
getConstructor : (k : Index cs) ->
  forall t. Pointer.Mu cs (k # t) ->
  IO (Pointer.Meaning (description k) cs t)
getConstructor (MkIndex k) mu
= let offs : Nat; offs = offsets (index k $ constructors cs) in
  getOffsets (muBuffer mu) (1 + muPosition mu) offs
$ let size = muSize mu - 1 - cast (8 * offs) in
  \ subterms, pos => MkMeaning subterms (muBuffer mu) pos size
```

To get a pointer to the meaning `t`, we start by getting the vector of offsets stored immediately after the tag. We then compute the size of the remaining `Meaning` description: it is the size of the overall tree, minus 1 (for the tag) and 8 times the number of offsets (because each offset is stored as an 8 bytes number). We can then use the record constructor `MkMeaning` to

pack together the vector of offsets, the buffer, the position past the offsets and the size we just computed.

```
getOffsets : Buffer -> (pos : Int) ->
  (n : Nat) ->
  forall t. (Vect n Int -> Int -> Pointer.Meaning d cs t) ->
  IO (Pointer.Meaning d cs t)
getOffsets buf pos 0 k = pure (k [] pos)
getOffsets buf pos (S n) k = do
  off <- getInt buf pos
  getOffsets buf (8 + pos) n (k . (off ::))
```

The implementation of `getOffsets` is straightforward: given a continuation that expect `n` offsets as well as the position past the last of these offsets, we read the 8-bytes-long offsets one by one and pass them to the continuation, making sure that we move the current position by 8 bytes before every recursive call.

We can readily use this function to inspect a top-level constructor in a correct-by-construction manner like in the following example.

<pre>isLeaf : Mu Tree -> Bool isLeaf ("Leaf" # _) = True isLeaf _ = False</pre>	<pre>isLeaf : Pointer.Mu Tree t -> IO (Singleton (isLeaf t)) isLeaf p = out p >>= \case "Leaf" # _ => pure [True] "Node" # _ => pure [False]</pre>
--	---

Re-using the `Tree` description we introduced in [Section 2.4](#), we defined a pure `isLeaf` function checking that an inductive tree is a leaf, together with its effectful equivalent using `out` to reveal the tree's top node thus allowing the type-level call to the pure `isLeaf` to reduce in each of the lambda-case's branches. The use of `Singleton` guarantees that we indeed return the appropriate boolean for the tree we are pointing to.

8.4 Offering a Convenient View

We can combine `out` and `layer` to obtain the `view` function we used in our introductory examples in [Section 1.1](#). This operation is not part of the trusted core: provided that `poke` and `out` are correct then it will automatically be correct-by-construction. A `(View cs t)` value gives us access to the `(Index cs)` of `t`'s top constructor together with the corresponding `Layer` of deserialised values and pointers to subtrees.

```
data View : (cs : Data nm) -> (t : Data.Mu cs) -> Type where
  (#) : (k : Index cs) ->
    forall t. Layer (description k) cs t ->
    View cs (k # t)
```

The implementation of `view` is unsurprising: we use `out` to expose the top constructor index and a `Pointer.Meaning` to the constructor's payload. We then use `layer` to extract the full `Layer` of deserialised values that the pointer references.

```

view : {cs : Data nm} -> forall t.
  Pointer.Mu cs t -> IO (View cs t)
view ptr = do k # el <- out ptr
  vs <- layer el
  pure (k # vs)

```

It is worth noting that although a `view` may be convenient to consume, a performance-minded user may decide to directly use the `out` and `poke` combinators to avoid deserialising values that they do not need. We present a case study in [Appendix 2](#) comparing the access patterns of two implementations of the function fetching the byte stored in a tree's rightmost node depending on whether we use `view` or the lower level `poke` combinator.

By repeatedly calling `view`, we can define the correct-by-construction generic deserialisation function: by using `Singleton`, its type guarantees that we turn a pointer to a tree `t` into a runtime value known to be equal to `t`.

```

deserialise : {cs : Data nm} -> forall t.
  Pointer.Mu cs t -> IO (Singleton t)

```

We can measure the benefits of our approach by comparing the runtime of a function directly operating on buffers to its pure counterpart composed with a deserialisation step. For functions like `rightmost` that only explore a very small part of the full tree, the gains are spectacular: the process operating on buffers is exponentially faster than its counterpart which needs to deserialise the entire tree first (cf. [Section 10](#)).

8.5 Generic Fold

The implementation of the generic `fold` over a tree stored in a buffer is going to have the same structure as the generic fold over inductive values: first match on the top constructor, then use `fmap` to apply the fold to all the substructures and, finally, apply the algebra to the result. We start by implementing the buffer-based counterpart to `fmap`. Let us go through the details of its type first.

```

fmap : (d : Desc r s o) ->
  (0 f : Data.Mu cs -> b) ->
  (forall t. Pointer.Mu cs t -> IO (Singleton (f t))) ->
  forall t. Pointer.Meaning d cs t -> IO (Singleton (Data.fmap d f t))

```

The first two arguments to `fmap` are similar to its pure counterpart: a description `d` and a (here runtime-irrelevant) function `f` to map over a `Meaning`. Next we take a function which is the buffer-aware counterpart to `f`: given any runtime-irrelevant term `t` and a pointer to it in a buffer, it returns an `IO` process computing the value `(f t)`. Finally, we take a runtime-irrelevant meaning `t` as well as a pointer to its representation in a buffer and compute an `IO` process which will return a value equal to `(Data.fmap d f t)`. We can now look at the definition of `fmap`.

```

fmap d f act ptr = poke ptr >>= go d where
go : (d : Desc{}) -> forall t. Poke d cs t ->
  IO (Singleton (Data.fmap d f t))
go None I = pure [| I |]
go Byte v = pure v
go (Prod d e) (v # w)
  = do fv <- fmap d f act v
      fw <- fmap e f act w
      pure [| fv # fw |]
go Rec v = act v

```

We poke the buffer to reveal the value the `Pointer.Meaning` named `ptr` is pointing at and then dispatch over the description `d` using the `go` auxiliary function. If the description is `None` we match on the constructor `I` of the `Poke`' which refines the type-level term `t` to the constructor `I` of the record `True` and thus allows the `fmap` call to reduce to `I`. We can therefore immediately return the singleton wrapping the value `I`. If the description is `Byte`, the value is left untouched and so we can simply return it immediately. If we have a `Prod` of two descriptions, we recursively apply `fmap` to each of them and pair the results back. Finally, if we have a `Rec` we apply the function operating on buffers that we know performs the same computation as `f`.

We can now combine `out` and `fmap` to compute the correct-by-construction `fold`: provided an algebra for a datatype `cs` and a pointer to a tree `t` of type `cs` stored in a buffer, we return an `IO` process computing the same value as the pure `fold` would have when applied to `t`.

```

fold : {cs : Data nm} -> (alg : Alg cs a) ->
  forall t. Pointer.Mu cs t ->
  IO (Singleton (Data.fold alg t))

```

We first use `out` to reveal the constructor choice in the tree's top node, we then recursively apply `(fold alg)` to all the substructures by calling `fmap`, and we conclude by applying the algebra to this result.

```

fold alg ptr
  = do k # t <- out ptr
      rec <- assert_total (fmap _ _ (fold alg) t)
      pure (alg k <$> rec)

```

We once again (cf. [Section 3.3](#)) had to use `assert_total` because it is not obvious to Idris 2 that `fmap` only uses its argument on subterms. This could have also been avoided by mutually defining `fold` and a specialised version of `(fmap (fold alg))` at the cost of code duplication and obfuscation. We once again include such a definition in [Appendix 1](#).

9 Serialising Data

So far all of our example programs involved taking an inductive value apart and computing a return value in the host language. But we may instead want to compute another value in serialised form. We include below one such example: a `map` function which takes a function `f` acting on bytes, a pointer to a `Tree` named `t` and returns a serialisation process that will eventually produce another `Tree` equal to the one obtained by applying that function all of the bytes stored in `t`'s nodes.

```
map : (f : Bits8 -> Bits8) -> (ptr : Pointer.Mu Tree t) ->
      Serialising Tree (Data.map f t)
map f ptr = case !(view ptr) of
  "Leaf" # l => "Leaf" # l
  "Node" # l # b # r => "Node" # map f l # [| f b |] # map f r
```

It calls the `view` we just defined to observe whether the tree is a leaf or a node. If it's a leaf, it returns a leaf. If it's a node, it returns a node where the `map` has been recursively applied to the left and right subtrees while the function `f` has been applied to the byte `b`.

In this section we are going to spell out how we can define the high-level constructs used above to allow users to write these correct-by-construction serialisers.

9.1 The Type of Serialisation Processes

A serialisation process for a tree `t` that belongs to the datatype `cs` is a function that takes a buffer and a starting position and returns an `IO` process that serialises the term in the buffer at that position and computes the position of the first byte past the serialised tree.

```
record Serialising (cs : Data nm) (t : Data.Mu cs) where
  constructor MkSerialising
  runSerialising : Buffer -> Int -> IO Int
```

We do not expect users to define such processes by hand and in fact prevent them from doing so by not exporting the `MkSerialising` constructor. Instead, we provide high-level, invariant-respecting combinators to safely construct such serialisation processes.

9.2 Building Serialisation Processes

Our main combinator is `(#)`: by providing a node's constructor index and a way to serialise all of the node's subtrees, we obtain a serialisation process for said node. We will give a detailed explanation of `All` below.

```
(#) : {cs : Data nm} -> (k : Index cs) ->
      {0 t : Meaning (description k) (Data.Mu cs)} ->
      All (description k) (Serialising cs) t ->
      Serialising cs (k # t)
```

The keen reader may refer to the accompanying code to see the implementation. Informally (cf. [Section 4.2](#) for the description of the format): first we write the tag corresponding to the choice of constructor, then we leave some space for the offsets, in the meantime we write all of the constructor's arguments and collect the offsets associated to each subtree while doing so, and finally we fill in the space we had left blank with the offsets we have thus collected.

The **All** quantifier performs the pointwise lifting of a predicate over the functor described by a **Desc**. It is defined by induction over the description.

```
All : (d : Desc r s o) -> (p : x -> Type) -> Meaning d x -> Type
All Byte p t = Singleton t
All Rec p t = p t
All d p t = All' d p t
```

If the description is **Byte** we only demand that we have a runtime copy of the byte so that we may write it inside a buffer. This is done using the **Singleton** family discussed in [Section 7.2](#). If the description is **Rec** then we demand that the predicate holds. If the description is either **None** or **Prod** then we use once again an auxiliary family purely for ergonomics. It is defined mutually with **All** and does the expected structural operation.

```
data All' : (d : Desc r s o) -> (p : x -> Type) ->
    Meaning d x -> Type where
  I : All' None p I
  (#) : All d p t -> All e p u -> All' (Prod d e) p (t # u)
```

It should now be clear that (**All** (**description** **k**) (**Serialising** **cs**)) indeed corresponds to having already defined a serialisation process for each subtree.

This very general combinator should be enough to define all the serialisers we may ever want. By repeatedly pattern-matching on the input tree and using (**#**), we can for instance define the correct-by-construction generic serialisation function.

```
serialise : {cs : Data nm} -> (t : Data.Mu cs) -> Serialising cs t
```

We nonetheless include other combinators purely for performance reasons.

9.3 Copying Entire Trees

We introduce a **copy** combinator for trees that we want to serialise as-is and have a pointer for. Equipped with this combinator, we are able to easily write e.g. the **swap** function which takes a binary tree apart and swaps its left and right branches (if the tree is non-empty).

```
swap : Pointer.Mu Tree t ->
    Serialising Tree (Data.swap t)
swap ptr = case !(view ptr) of
  "Leaf" # I => "Leaf" # I
  "Node" # l # b # r =>
    "Node" # copy r # b # copy l
```

We could define this `copy` combinator at a high level either by composing `deserialise` and `serialise`, or by interleaving calls to `view` and `(#)`. This would however lead to a slow implementation that needs to traverse the entire tree in order to simply copy it.

Instead, we implement `copy` by using the `copyData` primitive for `Buffers` present in Idris 2's standard library. This primitive allows us to grab a slice of the source buffer corresponding to the tree and to copy the raw bytes directly into the target buffer. This use of an unsafe primitive makes `copy` part of the trusted core for this library.

```
copy : Pointer.Mu cs t -> Serialising cs t
copy ptr = MkSerialising $ \ buf, pos => do
  let size = muSize ptr
  copyData (muBuffer ptr) (muPosition ptr) size buf pos
  pure (pos + size)
```

This is the one combinator that crucially relies on our format only using offsets and not absolute addresses and on the accuracy of the size information we have been keeping in `Pointer.Mu` and `Pointer.Meaning`. As we can see in [Section 10](#), this is spectacularly faster than a deep copying process traversing the tree.

9.4 Executing a Serialisation Action

Now that we can describe actions serialising a value to a buffer, the last basic building block we are still missing is a function actually performing such actions. This is provided by the `execSerialising` function declared below.

```
execSerialising : {cs : Data nm} -> {t : Data.Mu cs} ->
  Serialising cs t -> IO (Pointer.Mu cs t)
```

By executing a `(Serialising cs t)`, we obtain an `IO` process returning a pointer to the tree `t` stored in a buffer. We can then either compute further with this tree (e.g. by calling `sum` on it), or write it to a file for safekeeping using the function `writeToFile` introduced in [Section 6.2](#).

9.5 Evaluation Order

The careful reader may have noticed that we can and do run arbitrary `IO` operations when building a value of type `Serialising` (cf. the `map` example in [Section 9](#) where we perform a call to `view` to inspect the input's shape).

This is possible thanks to Idris 2 elaborating `do`-blocks using whichever appropriate bind operator is in scope. In particular, we have defined the following one to use when building a serialisation process:

```

(>>=) : IO a -> (a -> Serialising cs t) -> Serialising cs t
io >>= f = MkSerialising $ \buf, start =>
    do x <- io
      runSerialising (f x) buf start

```

By using this bind we can temporarily pause writing to the buffer to make arbitrary `IO` requests to the outside world. In particular, this allows us to interleave reading from the original buffer and writing into the target one thus having a much better memory footprint than if we were to first use the `IO` monad to build in one go the whole serialisation process for a given tree and then execute it.

10 Benchmarks

Now that we have the ability to read, write, and program directly over trees stored in a buffer we can run some experiments to see whether this allows us to gain anything over the purely functional programming style.

For all of these tests we generate full trees for each depth, labeling its nodes with the bytes 0, 1, etc. in a depth-first left-to-right manner.

Each test is run 25 times in a row, and the duration averaged. We manually run chezscheme's garbage collector before the start of each time measurement.

All of our plots use a logarithmic y axis because the runtime of the deserialisation-based functions is necessarily exponential in the depth of the full tree.

10.1 Pointer vs. Data

We will first look at the runtime characteristics of various functions implemented using the views defined in [Section 8](#). We compare the time it takes to run the composition of deserialising the tree and applying the pure function to the time it takes to run its pointer-based counterpart.

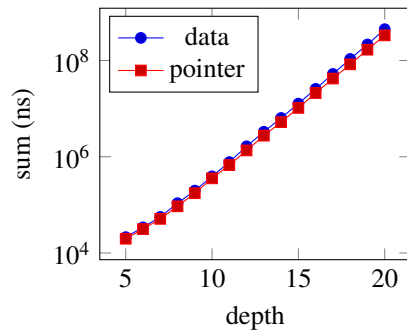
10.1.1 Traversing the Full Tree

The `sum` function visits all of the tree's nodes to add up all of the bytes that are stored.

```

sum : Data.Mu Tree -> Nat
sum t = case t of
  "Leaf" # _ => Z
  "Node" # l # b # r =>
    let m = sum l
        n = sum r
    in (m + cast b + n)

```

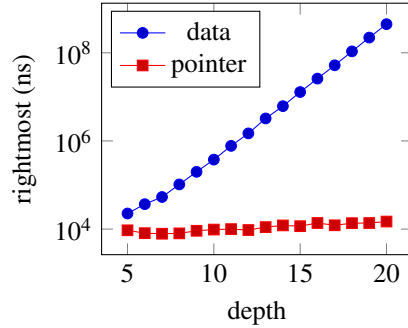


Because this function explores the entirety of the tree, the difference between the deserialisation-based and the pointer-based functions is minimal.

10.1.2 Skipping Most of the Tree

The `rightmost` function goes down the tree's right hand and returns the byte stored in its rightmost node.

```
rightmost : Data.Mu Tree ->
            Maybe Bits8
rightmost t = case t of
  "Leaf" # _ => Nothing
  "Node" # l # b # r =>
    let mval = rightmost r in
    Just (fromMaybe b mval)
```



Because this function only traverses the rightmost branch of the tree, the pointer based implementation is able to run in linear time by efficiently skipping past every left subtree. This is effectively an exponential speedup compared to the implementation that first fully deserialises the tree.

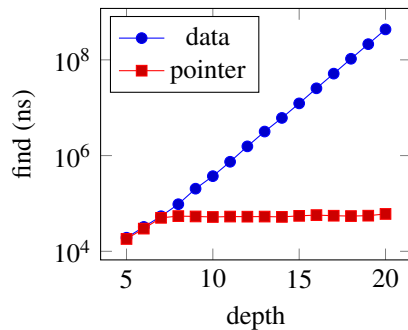
10.1.3 Exploring a Bounded Fragment of the Tree

Let us for a change look at a function returning a richly typed result. The `find` function looks for a given byte in a tree and returns a path to it if it can. The notion of `Path` is defined inductively: if the `tgt` byte we are looking for is in the node then we can use `Here`; otherwise we can take a turn down the left or right subtree using `TurnL` and `TurnR` respectively and provide a path to the target byte in the corresponding subtree.

```
data Path : Bits8 -> Data.Mu Tree -> Type where
  Here : Path tgt (node l tgt r)
  TurnL : Path tgt l -> Path tgt (node l b r)
  TurnR : Path tgt r -> Path tgt (node l b r)
```

To save space we only present a semi-decision procedure but this could be generalised to a full decision procedure. Given that `Path` is a very informative type, in this test case the version manipulating pointers does not bother using the `Singleton` family.

```
find : (tgt : Bits8) ->
      (t : Data.Mu Tree) ->
      Maybe (Path tgt t)
find tgt t = case t of
  "Leaf" # _ => Nothing
  "Node" # l # b # r => do
    let No _ = decEq tgt b
    | Yes Refl => pure Here
    TurnL <$> find tgt l
    <|> TurnR <$> find tgt r
```



We run the test using the byte 120. Due to the way our test trees are generated, we will only need to explore at most 121 of the tree's nodes before finding this byte. Unsurprisingly

we observe that the pointer-based function is constant time while the one operating over data is exponential due to the deserialisation step.

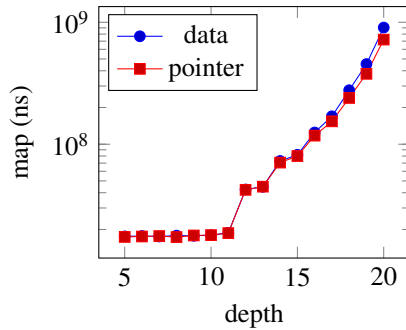
10.2 Serialising

Let us now turn to the time characteristics of the serialisation primitives defined in [Section 9](#). We compare the time it takes to run the composition of deserialising the tree, applying the pure function, and serialising the result to the time it takes to run its pointer-based counterpart using the [Serialising](#) framework.

10.2.1 Traversing the Full Tree

The `map` function applies a byte-to-byte function to all of the bytes stored in a tree's node. We run it using the `(+ 100)` function.

```
map : (f : Bits8 -> Bits8) ->
      (ptr : Pointer.Mu Tree t) ->
      Serialising Tree (Data.map f t)
map f ptr = case !(view ptr) of
  "Leaf" # I => "Leaf" # I
  "Node" # l # b # r =>
    "Node" # map f l # [l f b l]
              # map f r
```

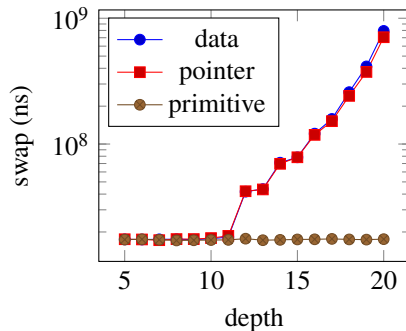


We can see that both approaches yield a similar runtime: everything is dominated by traversing the whole tree and building the resulting output. This is where a framework allowing for destructive updates may help write faster functions. This also has untapped opportunities for parallelism.

10.2.2 Using the Copy Primitive

The `swap` function takes a tree and, if it is non-empty, swaps its left and right subtrees. For this test case we compare an implementation using the *primitive copy* operator we introduced in [Section 9.3](#), one using a *pointer-based* copy that interleaves exposing the head constructor and serialising it, and a copy going via the *data* representation by fully deserialising the tree before re-serialising it.

```
swap : Pointer.Mu Tree t ->
      Serialising Tree (Data.swap t)
swap ptr = case !(view ptr) of
  "Leaf" # I => "Leaf" # I
  "Node" # l # b # r =>
    "Node" # copy r # b # copy l
```



We can see that, the *pointer* and *data* variants yield a similar runtime as they need to traverse the full tree whereas the primitive one based on copying raw buffer slices is significantly faster.

11 Conclusion

We have seen how, using a universe of descriptions indexed by their static and dynamic sizes, we can define a precise language of values serialised in a buffer. This allowed us to develop a library to manipulate such trees in a seamless, correct, and generic manner either using low-level combinators like *poke* or high-level programs like a data-polymorphic *fold*. We then provided users with convenient tools to write serialisation processes thus allowing them to compositionally build correct-by-construction values stored in buffers. Finally, we demonstrated that operating directly over serialised data can yield exponential speedups for functions that do not need to explore the whole tree they receive as an input.

11.1 Related Work

This work sits at the intersection of many domains: data-generic programming, the efficient runtime representation of functional data, programming over serialised values, and the design of serialisation formats. Correspondingly, a lot of related work is worth discussing. In many cases the advantage of our approach is precisely that it is at the intersection of all of these strands of research: we do it generically, seamlessly, in a correct-by-construction manner, and use actual buffers.

11.1.1 Data-Generic Programming

There is a long tradition of data-generic programming (Gibbons, 2006) and we will mostly focus here on the approach based on the careful reification of a precise universe of discourse as an inductive family in a host type theory, and the definition of generic programs by induction over this family.

One early such instance is Pfeifer and Rueß’ ‘polytypic proof construction’ (1999) meant to replace unsafe meta-programs deriving recursors (be they built-in support, or user-written tactics). Their approach requires the definition of an entirely new core language to support polytypic function definitions while ours is completely non-invasive. It would be interesting to port some of their motivating examples to our setting such as their generic data compression scheme for serialised tree representations.

In his PhD thesis, Morris (2007) declares various universes for strictly positive types and families and defines by generic programming further types (the type of one-hole contexts), modalities (the universal and existential predicate lifting over the functors he considers), and functions (map, boolean equality). We reuse some of these constructions already (e.g. the universal predicate lifting over functors is the *A11* type family we used to define serialisation combinators in Section 9) and the other ones are obvious candidates for future work. The type of one-hole contexts is in particular extremely useful to implement tail-recursive operations traversing a data structure.

Löh and Magalhães (2011) define a more expressive universe over indexed functors that is closed under composition and fixpoints. They also detail how to define additional generic construction such as a proof of decidable equality, various recursion schemes, and zippers. This work, quite similar to our own in its presentation, offers a natural candidate universe for us to use to extend our library.

Recent efforts by Escot and Cockx (2022) have demonstrated that it is possible, through reflection, to offer a neater user interface by transporting constructions between (a subset of) the native Agda inductive types and their rich universe of descriptions. This could potentially provide a more convenient way to program using our library: instead of having to define the functions in the specification layer over `Data.Mu` trees, users could program against bona fide host language datatypes while still benefiting from the proven-correct generic programs.

11.1.2 Efficient Runtime Representation of Inductive Values

Although not dealing explicitly with programming over serialised data, Monnier’s work (2019) with its focus on performance and in particular on the layout of inductive values at runtime, partially motivated our endeavour. Provided that we find a way to get the specialisation and partial evaluation of the generically defined views, we ought to be able to achieve –purely in user code– Monnier’s vision of a representation where n -ary tuples have constant-time access to each of their component.

Baudon, Radanne, and Gonnord (2023) adopted a radically different approach to ours: we provide users with a uniform representation that requires no setup on their part, while `RIBBIT` instead gives power users absolute freedom to define their own data layouts. The project provides seamless integration: users annotate their data declarations with the data layout they want. The programming experience is left unchanged: users write nested pattern-matching programs without ever needing to explicitly talk about the layout. They cover a universe of monomorphic immutable algebraic datatypes fairly similar to ours, however the annotations language is expressive enough to talk about e.g. principled struct packing, bit stealing, unboxing, untagged unions, or word alignment. Their approach is really impressive but requires modifying the whole compiler pipeline down to producing LLVM IR while ours lives purely in userland.

`DARGENT` (Chen et al., 2023) is a similar high-level framework empowering users to dictate the data layout that the generated C code shall use. This means that, just like in our project, users do not need to deserialise a value before being able to process it. This work has the added benefit, compared to `RIBBIT`, that it automatically generates an Isabelle/HOL correctness proof that the generated C code is a refinement of the original program.

Allais (2023) demonstrates how one can encode an invariant-rich inductive type using builtin types in Idris 2 and recover high level programming principles by using views. They rely heavily on Quantitative Type Theory to precisely control what parts of the encoding are to be erased during compilation. Their approach, dealing with a single inductive family in a tutorial-style paper, is ad-hoc but complementary to ours and shows how to use bit packing in a certified way.

11.1.3 Working on Serialised Data

Swierstra and van Geest (2017) define in Agda a rich universe of data descriptions and generically implement serialising and deserialising, proving that the latter is a left inverse to the former. Their universe is powerful enough that later parts of a description can be constrained using the *value* associated to earlier ones. We will be able to rely on this work when extending our current universe to one for type families. In turn, our approach could be ported to their setting to avoid the need to fully deserialise the data in order to process it.

LoCal (Vollmer et al., 2019) is the work that originally motivated the design of this library. We have demonstrated that generic programming within a dependently typed language can yield the sort of benefits other language can only achieve by inventing entirely new intermediate languages and compilation schemes.

LoCal was improved upon with a re-thinking of the serialisation scheme making the approach compatible with parallel programming (Koparkar et al., 2021). This impressive improvement is a natural candidate for future work on our part: the authors demonstrate it is possible to reap the benefits of both programming over serialised data and dividing up the work over multiple processors with almost no additional cost in the case of a purely sequential execution.

11.1.4 Serialisation Formats

The PADS project (Mandelbaum et al., 2007) aims to let users quickly, correctly, and compositionally describe existing formats they have no control over. As they remind us, ad-hoc serialisation formats abound be it in networking, logging, billing, or as output of measurement equipments in e.g. gene sequencing or molecular biology. Our current project is not offering this kind of versatility as we have decided to focus on a specific serialisation format with strong guarantees about the efficient access to subtrees. But our approach to defining correct-by-construction components could be leveraged in that setting too and bring users strong guarantees about the traversals they write.

ASN.1 (Larmouth, 1999) gives users the ability to define a high-level specification of the exchange format (the ‘abstract syntax’) to be used in communications without the need to concern themselves with the actual encoding as bit patterns (the ‘transfer syntax’). This separation between specification and implementation means that parsing and encoding can be defined once and for all by generic programming (here, a compiler turning specifications into code in the user’s host language of choice). The main difference is once again our ability to program in a correct-by-construction manner over the values thus represented.

Yallop’s automatic derivation of serialisers using an OCaml preprocessor (Yallop, 2007) highlights the importance of empowering domain experts to take advantage of the specifics of the problem they are solving to minimise the size of the encoded data. By detecting sharing using a custom equality function respecting α -equivalence instead of the default one, he was able to serialise large lambda terms using only a quarter of the bytes required by the OCaml’s standard library marshaller.

11.1.5 Type Theory for Data Layout

Petersen, Robert, Crary, and Pfenning (2003) design an elegant type theory based on ordered linear logic to describe the memory layout and an effectful functional language in the style of Moggy’s metalanguage (1991). This empowers users to carefully control their data’s memory layout. This logic-based approach supports the explicit reservation, initialisation, and allocation of new memory locations all without needing to explicitly talk about an underlying heap-based memory model. Their language’s intended semantics shares our economic flat structure but only for non-recursive data: components of a pair are stored contiguously, and values of the unit type are absent at runtime. In their extended technical report (2002) they describe support for sum types, both tagged and untagged (through an internalised notion of distinguishable types). They also describe a simple scheme to support iso-recursive types by giving up on a flat memory representation and insisting that every node is behind a pointer indirection. Their approach requires a dedicated language implementation while our definitions live purely in client code, defining an embedded domain specific language as a library. They support destructive memory updates but do not offer any support for internally proving the properties of the programs one defines.

11.2 Limitations and Future Work

Although our design is already proven to be functional by two implementations in Idris 2 and Agda respectively, we can always do better. In this section we are going to see what benefits future work could bring across the whole project.

11.2.1 A Smaller Trusted Core

Our current trusted core amounts to the implementations of `poke` (defined in Section 8.1), `out` (defined in Section 8.3), the serialisation combinator (`#`) (defined in Section 9.2), `copy` (defined in Section 9.3), and `execSerialising` (defined in Section 9.4).

As observed in Section 6 the indices used in `Pointer.Mu` and `Pointer.Meaning` are phantom indices (Leijen and Meijer, 1999). This is the reason why our arithmetic operations on pointers are unverified and we have to resort to escape hatches to deploy the observations made by reading bytes in the buffer in order to refine indices.

A further engineering effort could allow us to move this trusted core slightly down the stack by defining a type of buffers indexed by a (phantom) list of bytes corresponding to their content together with trusted reading primitives whose behaviour is expressed in terms of that phantom list. Reusing existing work on correct-by-construction serialisers (van Geest and Swierstra, 2017) we could then prove that the implementations of our basic building blocks are indeed safe provided that the buffer primitives are correct.

11.2.2 A More Robust Library

For sake of ease of presentation we have not dealt with issues necessitating buffer resizing: in Section 9, we defined `execSerialising` by allocating a fixed size buffer and not worrying whether the whole content would fit. A real library would need to adopt a more robust approach akin to the one used in the implementation of Idris 2’s own serialisation code:

whenever we are about to write a byte to the buffer, we make sure there is either enough space left or we grow it. Note that our Agda port does not suffer from this limitation as it can rely on Haskell’s `bytestring` library (Coumts et al., 2007) and use its `Builder` type.

In our library, the data types descriptions currently need to be defined as values in the host language. This opens up the opportunity for bugs if, say, we write a server in Idris 2 and a client in Agda and accidentally use two slightly different descriptions in the projects. This could be solved at the language level by equipping our dependently typed languages with dependent type providers like Idris 1 had (Christiansen, 2013). This way the format could be loaded at compile time from the same file thus ensuring all the components are referring to the exact same specification.

11.2.3 A More Efficient Library

Looking at the code generated by Idris 2, we notice that our generic programs are not specialised and partially evaluated even when the types they are working on are statically known. Refactoring the library to use a continuation-passing-style approach does help the compiler generate slightly more specialised code but the results are in our opinion not good enough to justify forcing users to program in this more cumbersome style. A possible alternative would be to present users with macros rather than generic programs so that the partial evaluation would be guaranteed to happen at typechecking time. This however makes the process of defining the generic programs much more error prone. A more principled approach would be to extend Idris 2 with a proper treatment of staging e.g. by using a two-level type theory as suggested by Kovács (2022).

Our serialisation format has been designed to avoid pointer-chasing and thus ensures entire subtrees can be easily copied by using the raw bytes. Correspondingly it currently does not support sharing. This could however be a crucial feature for trees with a lot of duplicated nodes and we would like to allow users to, using the same interface, easily pick between different serialisation formats so that the library ends up using the one that suits their application best. To this end, we could take inspiration from Yallop’s definition of preprocessors generating serialisers (Yallop, 2007). It maintains an object map containing the already serialised nodes and uses it to maximally detect sharing and maintain it both when serialising and deserialising.

Our current approach allows us to define a correct-by-construction `sum` operating directly on serialised data but it does not eliminate the call stack used in the naïve functional implementation. Converting a fold to a tail recursive function in a generic manner is a well studied problem and the existing solutions (McBride, 2008; Tomé Cortiñas and Swierstra, 2018) should be fairly straightforward, if time-consuming, to port to our setting.

11.2.4 A More Expressive Universe of Descriptions

We have used a minimal universe to demonstrate our approach but a practical application would require the ability to store more than just raw bytes. An easy extension is to add support for all of the numeric types of known size that Idris 2 offers (`Bits{8, 16, 32, 64}`, `Int{8, 16, 32, 64}`), for `Bool` as well as an unbounded data such as `Nat`, or `String` as long as an extra offset is provided for each value.

The storage of values smaller than a byte (here `Bool`) naturally raises the question of bit packing: why store eight booleans as eight bytes when they could fit in a single one? Our recent work (Allais, 2023) on the efficient runtime representation of inductive families as values of Idris 2’s primitive types points us in the direction of a solution.

A natural next candidate is a universe allowing the definition of parametrised types (Löh and Magalhães, 2011): we should be able to implement functions over arbitrary (`List a`) values stored in a buffer, provided that we know that `a` is serialisable. This was already an explicit need in ASN.1 (Larmouth, 1999), reflecting that protocols often leave ‘holes’ where the content of the protocol’s higher layer is to be inserted.

Next, we will want to consider a universe of indexed data: we can currently natively model algebraic datatypes such as lists or trees, and we can use the host language to compute the description of vectors by induction on their length, but we cannot model arbitrary type families (Dybjer, 1994) e.g. correct-by-construction red-black trees.

Last but not least we may want to have a universe of descriptions closed under least fixpoints (Morris, 2007) in order to represent rose trees for instance.

11.2.5 A More Expressive Library

Using McBride’s generalisation of one hole contexts (McBride, 2008) we ought to be able to give a more precise type to the combinator (`#`) used to build serialisation processes. When defining the serialisation of a given subtree, we ought to have access to pointers to the result of serialising any subtree to the left of it. In particular this would make building complete binary trees a lot faster by allowing us to rely on `copyData` for duplicating branches rather than running the computation twice.

Last, but not least we currently do not support in-place updates to the data stored in a buffer. This could however be beneficial for functions like `map`. It remains to be seen whether we can somehow leverage Idris 2’s linear quantity annotation to provide users with serialised value that can be safely updated in place. This would turn our ongoing metaphor involving Hoare triples (Hoare, 1969), heap pointers, and separation logic (Reynolds, 2002) into a bona fide shallow embedding. Poulsen, Rouvoet, Tolmach, Krebbers, and Visser’s pioneering work (Poulsen et al., 2018; Rouvoet, 2021) on definitional interpreters for languages with references and the use of a shallowly embedded separation logic to minimise bookkeeping give us a clear set of techniques to adapt to our setting.

Acknowledgements

We would like to thank Wouter Swierstra, Fredrik Nordvall-Forsberg, as well as the anonymous reviewers for their helpful comments on various drafts of this paper.

This research was partially funded by the Engineering and Physical Sciences Research Council (grant number EP/T007265/1).

References

Abel, A. & Pientka, B. (2011) Higher-order dynamic pattern unification for dependent types and records. Typed Lambda Calculi and Applications - 10th International Conference, TLCA 2011,

- Novi Sad, Serbia, June 1-3, 2011. Proceedings. Springer. pp. 10–26.
- Allais, G. (2023) Builtin types viewed as inductive families. Programming Languages and Systems - 32nd European Symposium on Programming, ESOP 2023, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2023, Paris, France, April 22-27, 2023, Proceedings. Springer. pp. 113–139.
- Altenkirch, T. & McBride, C. (2002) Generic programming within dependently typed programming. Generic Programming, IFIP TC2/WG2.1 Working Conference on Generic Programming, July 11-12, 2002, Dagstuhl, Germany. Kluwer. pp. 1–20.
- Atkey, R. (2018) Syntax and semantics of quantitative type theory. Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2018, Oxford, UK, July 09-12, 2018. ACM. pp. 56–65.
- Baudon, T., Radanne, G. & Gonnord, L. (2023) Bit-stealing made legal: Compilation for custom memory representations of algebraic data types. *Proc. ACM Program. Lang.* **7**(ICFP).
- Benke, M., Dybjer, P. & Jansson, P. (2003) Universes for generic programs and proofs in dependent type theory. *Nord. J. Comput.* **10**(4), 265–289.
- Brady, E. C. (2021) Idris 2: Quantitative type theory in practice. 35th European Conference on Object-Oriented Programming, ECOOP 2021, July 11-17, 2021, Aarhus, Denmark (Virtual Conference). Schloss Dagstuhl - Leibniz-Zentrum für Informatik. pp. 9:1–9:26.
- Charguéraud, A. (2023) *A Modern Eye on Separation Logic for Sequential Programs*. University of Strasbourg.
- Charguéraud, A. (2023) *Separation Logic Foundations*. vol. 6 of *Software Foundations*. Electronic textbook. Version 1.6.
- Chen, Z., Lafont, A., O'Connor, L., Keller, G., McLaughlin, C., Jackson, V. & Rizkallah, C. (2023) Dargent: A silver bullet for verified data layout refinement. *Proc. ACM Program. Lang.* **7**(POPL), 1369–1395.
- Christiansen, D. R. (2013) Dependent type providers. Proceedings of the 9th ACM SIGPLAN workshop on Generic programming, WGP 2013, Boston, Massachusetts, USA, September 28, 2013. ACM. pp. 25–34.
- Coutts, D., Stewart, D. & Leshchinskiy, R. (2007) Rewriting haskell strings. Practical Aspects of Declarative Languages, 9th International Symposium, PADL 2007, Nice, France, January 14-15, 2007. Springer. pp. 50–64.
- Dybjer, P. (1994) Inductive families. *Formal Aspects Comput.* **6**(4), 440–465.
- Escot, L. & Cockx, J. (2022) Practical generic programming over a universe of native datatypes. *Proc. ACM Program. Lang.* **6**(ICFP), 625–649.
- Gibbons, J. (2006) Datatype-generic programming. Datatype-Generic Programming - International Spring School, SSDGP 2006, Nottingham, UK, April 24-27, 2006, Revised Lectures. Springer. pp. 1–71.
- Hagino, T. (1987) A typed lambda calculus with categorical type constructors. Category Theory and Computer Science, Edinburgh, UK, September 7-9, 1987, Proceedings. Springer. pp. 140–157.
- Hoare, C. A. R. (1969) An axiomatic basis for computer programming. *Commun. ACM.* **12**(10), 576–580.
- Idris Community. (2023) Katla – LaTeX & HTML code listing generator for Idris 2. <https://github.com/idris-community/katla>.
- Jansson, P. & Jeuring, J. (1997) Polyp - A polytypic programming language. Conference Record of POPL'97: The 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Papers Presented at the Symposium, Paris, France, 15-17 January 1997. ACM Press. pp. 470–482.
- Koparkar, C., Rainey, M., Vollmer, M., Kulkarni, M. & Newton, R. R. (2021) Efficient tree-traversals: reconciling parallelism and dense data representations. *Proc. ACM Program. Lang.* **5**(ICFP), 1–29.
- Kovács, A. (2022) Staged compilation with two-level type theory. *Proc. ACM Program. Lang.* **6**(ICFP), 540–569.
- Larmouth, J. (1999) *ASN.1 Complete*. Morgan Kaufmann Publishers Inc. San Francisco, CA, USA.
- Leijen, D. & Meijer, E. (1999) Domain specific embedded compilers. Proceedings of the Second

- Conference on Domain-Specific Languages (DSL '99), Austin, Texas, USA, October 3-5, 1999. ACM. pp. 109–122.
- Löh, A. & Magalhães, J. P. (2011) Generic programming with indexed functors. Proceedings of the seventh ACM SIGPLAN workshop on Generic programming, WGP@ICFP 2011, Tokyo, Japan, September 19-21, 2011. ACM. pp. 1–12.
- Malcolm, G. (1990) Data structures and program transformation. *Sci. Comput. Program.* **14**(2-3), 255–279.
- Mandelbaum, Y., Fisher, K., Walker, D., Fernández, M. F. & Gleyzer, A. (2007) PADS/ML: a functional data description language. Proceedings of the 34th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2007, Nice, France, January 17-19, 2007. ACM. pp. 77–83.
- McBride, C. (2008) Clowns to the left of me, jokers to the right (pearl): dissecting data structures. Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2008, San Francisco, California, USA, January 7-12, 2008. ACM. pp. 287–295.
- McBride, C. (2016) I got plenty o' nuttin'. A List of Successes That Can Change the World - Essays Dedicated to Philip Wadler on the Occasion of His 60th Birthday. Springer. pp. 207–233.
- McBride, C. & McKinna, J. (2004) The view from the left. *J. Funct. Program.* **14**(1), 69–111.
- McBride, C. & Paterson, R. (2008) Applicative programming with effects. *J. Funct. Program.* **18**(1), 1–13.
- Mendel-Gleason, G. (2012) *Types and verification for infinite state systems*. Ph.D. thesis. Dublin City University.
- Moggi, E. (1991) Notions of computation and monads. *Inf. Comput.* **93**(1), 55–92.
- Monnier, S. (2019) Inductive types deconstructed: the calculus of united constructions. Proceedings of the 4th ACM SIGPLAN International Workshop on Type-Driven Development, TyDe@ICFP 2019, Berlin, Germany, August 18, 2019. ACM. pp. 52–63.
- Morris, P. W. J. (2007) *Constructing Universes for Generic Programming*. Ph.D. thesis. University of Nottingham, UK.
- O'Hearn, P. W. (2019) Separation logic. *Commun. ACM.* **62**(2), 86–95.
- Petersen, L., Harper, R., Crary, K. & Pfenning, F. (2002) A type theory for memory allocation and data layout (extended version). Technical Report CMU-CS-02-171.
- Petersen, L., Harper, R., Crary, K. & Pfenning, F. (2003) A type theory for memory allocation and data layout. Conference Record of POPL 2003: The 30th SIGPLAN-SIGACT Symposium on Principles of Programming Languages, New Orleans, Louisiana, USA, January 15-17, 2003. ACM. pp. 172–184.
- Pfeifer, H. & Rueß, H. (1999) Polymorphic proof construction. Theorem Proving in Higher Order Logics, 12th International Conference, TPHOLs'99, Nice, France, September, 1999, Proceedings. Springer. pp. 55–72.
- Poulsen, C. B., Rouvoet, A., Tolmach, A., Krebbers, R. & Visser, E. (2018) Intrinsically-typed definitional interpreters for imperative languages. *Proc. ACM Program. Lang.* **2**(POPL), 16:1–16:34.
- Reynolds, J. C. (2002) Separation logic: A logic for shared mutable data structures. 17th IEEE Symposium on Logic in Computer Science (LICS 2002), 22-25 July 2002, Copenhagen, Denmark, Proceedings. IEEE Computer Society. pp. 55–74.
- Rouvoet, A. (2021) *Correct by Construction Language Implementations*. Ph.D. thesis. Delft University of Technology, Netherlands.
- Tomé Cortiñas, C. & Swierstra, W. (2018) From algebra to abstract machine: a verified generic construction. Proceedings of the 3rd ACM SIGPLAN International Workshop on Type-Driven Development, TyDe@ICFP 2018, St. Louis, MO, USA, September 27, 2018. ACM. pp. 78–90.
- van Geest, M. & Swierstra, W. (2017) Generic packet descriptions: verified parsing and pretty printing of low-level data. Proceedings of the 2nd ACM SIGPLAN International Workshop on Type-Driven Development, TyDe@ICFP 2017, Oxford, UK, September 3, 2017. ACM. pp. 30–40.
- Vollmer, M., Koparkar, C., Rainey, M., Sakka, L., Kulkarni, M. & Newton, R. R. (2019) LoCal: a language for programs operating on serialized data. Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2019, Phoenix, AZ,

USA, June 22-26, 2019. ACM. pp. 48–62.

Wadler, P. (1987) Views: A way for pattern matching to cohabit with data abstraction. Conference Record of the Fourteenth Annual ACM Symposium on Principles of Programming Languages, Munich, Germany, January 21-23, 1987. ACM Press. pp. 307–313.

Yallop, J. (2007) Practical generic programming in ocaml. Proceedings of the ACM Workshop on ML, 2007, Freiburg, Germany, October 5, 2007. ACM. pp. 83–94.

1 Safe Implementations of fold

We include below the alternative definitions of `fold` (respectively processing inductive data and data stored in a buffer) which are seen as total by Idris 2. Each of them is mutually defined with what is essentially the supercompilation of `(\ d => fmap d (fold alg))`.

```
parameters {cs : Data nm} (alg : Alg cs a)

fold : Data.Mu cs -> a
fmapFold : (d : Desc{}) ->
    Data.Meaning d (Data.Mu cs) -> Data.Meaning d a

fold (k # t) = alg k (fmapFold (description k) t)

fmapFold None t = t
fmapFold Byte t = t
fmapFold (Prod d e) (s # t)
    = (fmapFold d s # fmapFold e t)
fmapFold Rec t = fold t
```

```

parameters {cs : Data nm} (alg : Alg cs a)

fold : Pointer.Mu cs t -> IO (Singleton (fold alg t))
fmapFold : (d : Desc{}) -> forall t. Pointer.Meaning d cs t ->
  IO (Singleton (fmapFold alg d t))

fold ptr
  = do k # t <- out ptr
      rec <- fmapFold (description k) t
      pure (alg k <$> rec)

fmapFold d ptr = poke ptr >=> go d where

go : (d : Desc{}) -> forall t. Poke d cs t ->
  IO (Singleton (fmapFold alg d t))
go None I = pure (pure I)
go Byte v = pure v
go (Prod d e) (v # w)
  = do v <- fmapFold d v
      w <- fmapFold e w
      pure [| v # w |]
go Rec v = fold v

```

2 Access Patterns: Viewing vs. Poking

In this example we implement `rightmost`, the function walking down the rightmost branch of our type of binary trees and returning the content of its rightmost node (if it exists).

The first implementation is the most straightforward: use `view` to obtain the top constructor as well as an entire layer of deserialised values and pointers to substructures and inspect the constructor. If we have a leaf then there is no byte to return. If we have a node then call `rightmost` recursively and inspect the result: if we got `Nothing` back we are at the rightmost node and can return the current byte, otherwise simply propagate the result.

```

rightmost : Pointer.Mu Tree t -> IO (Maybe Bits8)
rightmost ptr = case !(view ptr) of
  "Leaf" # _ => pure Nothing
  "Node" # _ # b # r => do
    mval <- rightmost r
    case mval of
      Just _ => pure mval
      Nothing => pure (Just (getSingleton b))

```

In the alternative implementation we use `out` to expose the top constructor and then, in the node case, call `poke` multiple times to get our hands on the pointer to the right subtree. We inspect the result of recursively calling `rightmost` on this subtree and only deserialise the byte contained in the current node if the result we get back is `Nothing`.

```

rightmost : Pointer.Mu Tree t -> IO (Maybe Bits8)
rightmost ptr = case !(out ptr) of
  "Leaf" # _ => pure Nothing
  "Node" # el => do
    (_ # br) <- poke el
    (b # r) <- poke br
    mval <- rightmost !(poke r)
    case mval of
      Just _ => pure mval
      Nothing => do
        b <- poke b
        pure (Just (getSingleton b))

```

This will give rise to two different access patterns: the first function will have deserialised all of the bytes stored in the nodes along the tree's rightmost path whereas the second will only have deserialised the rightmost byte. Admittedly deserialising a byte is not extremely expensive but in a more realistic example we could have for instance been storing arbitrarily large values in these nodes. In that case it may be worth trading convenience for making sure we are not doing any unnecessary work.